

BacktestBench: Benchmarking Large Language Models for Automated Quantitative Strategy Backtesting

Zhensheng Wang
Beijing Normal University
Beijing, China
jensenwang@mail.bnu.edu.cn

Wenmian Yang*
Beijing Normal University
Zhuhai, China
wenmianyang@bnu.edu.cn

Qingtai Wu
Beijing Normal University
Zhuhai, China
qingtaiwu@mail.bnu.edu.cn

Lequan Ma
Beijing Normal University
Zhuhai, China
lequanma@mail.bnu.edu.cn

Yiquan Zhang
Elmleaf Ltd.
Shanghai, China
zhangyq987@hotmail.com

Weijia Jia*
Beijing Normal University
Zhuhai, China
jiawj@bnu.edu.cn

Abstract

Quantitative backtesting is essential for evaluating trading strategies but remains hampered by high technical barriers and limited scalability. While Large Language Models (LLMs) offer a transformative path to automate this complex, interdisciplinary workflow through advanced code generation, tool usage, and agentic planning, the practical realization is significantly challenged by the current lack of a large-scale benchmark dedicated to automated quantitative backtesting, which hinders progress in this field. To bridge this critical gap, we introduce BacktestBench, the first large-scale benchmark for automated quantitative backtesting. Built from over 6 million real market records, it comprises 18,246 meticulously annotated question-answering pairs across four task categories: metrics calculation, ticker selection, strategy selection, and parameter confirmation. We also propose AutoBacktest, a robust multi-agent baseline that translates natural language strategies into reproducible backtests by coordinating a Summarizer for semantic factor extraction, a Retriever for validated SQL generation, and a Coder for Python backtesting implementation. Our evaluation on 23 mainstream LLMs, complemented by targeted ablations, identifies key factors that influence end-to-end performance and highlights the importance of grounded verification and standardized indicator representations. The dataset and code will be publicly released at <https://github.com/jensenw1/BacktestBench>.

CCS Concepts

• **Computing methodologies** → **Language resources**; **Multi-agent systems**; • **Information systems** → **Question answering**.

Keywords

Large Language Models, Question Answering, Quantitative Investing, Automated Backtesting, Benchmark, Multi-agent Framework

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD 2026, Jeju Island, Republic of Korea.*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817460>

ACM Reference Format:

Zhensheng Wang, Wenmian Yang, Qingtai Wu, Lequan Ma, Yiquan Zhang, and Weijia Jia. 2026. BacktestBench: Benchmarking Large Language Models for Automated Quantitative Strategy Backtesting. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 25 pages. <https://doi.org/10.1145/3770855.3817460>

1 Introduction

“History does not repeat itself, but it often rhymes.”

— Mark Twain

In quantitative investing, this aphorism captures the fundamental role of backtesting: it involves replaying trading logic on extensive historical data to rigorously assess strategy robustness and its potential adaptability to future market regimes [19]. By reporting quantitative metrics such as the sharpe ratio [22] and maximum drawdown [17], backtesting provides an objective evaluation framework, significantly reducing reliance on subjective judgment and enhancing comparability across diverse investment strategies [5]. However, the traditional backtesting workflow, as illustrated in Figure 1, is inherently complex and interdisciplinary, requiring specialized expertise. Strategy engineers must precisely construct factor combinations, issue accurate historical data queries for specific tickers and time windows, and implement error-free code to execute intricate strategy logic. This process is constrained by high technical barriers, demanding meticulous data retrieval, strict adherence to market timing, and precise trading rule implementation.

The recent advent of Large Language Models (LLMs) and their advanced capabilities in code generation, logical reasoning [3, 16, 20], and, more importantly, agentic planning [32], tool invocation [21], and iterative refinement [23] in interactive environments, presents a transformative opportunity for automating quantitative backtesting. Faced with an exploding search space of factor combinations and rapidly shifting market demands, manual strategy implementation and verification processes can no longer meet the efficiency and scalability requirements of modern quantitative research. This context prompts us to investigate whether LLM-based agents can reliably automate the entire backtesting pipeline, encompassing data retrieval, strategy translation, and execution. Such an endeavor also serves as a practical touchstone for evaluating the capability of Artificial General Intelligence (AGI) [27] on complex vertical-domain problems.



Figure 1: Real-world workflow of a quantitative backtesting engineer.

While the prospect of natural language-driven end-to-end automated backtesting is highly promising, its practical realization presents significant challenges. It demands that models simultaneously possess precise semantic understanding of financial strategies, robust structured data retrieval capabilities, and highly accurate code generation skills [30]. Critically, the academic community currently lacks high-quality evaluation benchmarks specifically designed for such complex financial reasoning tasks. Existing code generation or Text-to-SQL datasets [13, 14, 34], while valuable, do not adequately capture the unique temporal logic and multi-step reasoning requirements inherent in quantitative backtesting.

To bridge this critical gap, we introduce BacktestBench, a novel large-scale benchmark dataset tailored for the field of automated backtesting. Constructed from over 6 million real historical market records from China’s three major exchanges, BacktestBench comprises 18,246 high-quality question-answering (QA) pairs. Each sample is meticulously annotated, covering the entire information chain from a natural language strategy description to core factor extraction, SQL data query generation, Python backtesting code implementation, and the final objective answer. Furthermore, to comprehensively evaluate the holistic capabilities of models, this benchmark systematically designs four distinct task categories: metrics calculation, ticker selection, strategy selection, and parameter confirmation. These categories fully encompass real-world backtesting scenarios such as strategy comparison, optimization, and asset allocation, thereby providing a rigorous yardstick for measuring the intelligence level of LLMs in this vertical domain.

To systematically address BacktestBench’s challenges and effectively evaluate LLMs’ reasoning and execution capabilities in this domain, this study proposes AutoBacktest, a multi-agent collaborative system serving as a robust baseline. It transforms unstructured natural language strategy descriptions into reproducible backtest results, mimicking human quantitative researchers’ workflows by decoupling complex decision chains into specialized sub-tasks. Specifically, AutoBacktest coordinates three functionally specialized agents: (1) the Summarizer, responsible for semantic-level extraction of financial indicators (including both trading factors and key performance indicators); (2) the Retriever, which handles data-level precise querying and quality verification; and (3) the Coder, focusing on logic-level code implementation and backtest execution. This layered, modular architecture not only significantly reduces the inherent difficulty of end-to-end generation but also enhances the robustness of the entire backtesting pipeline through

self-verification mechanisms at each stage, thereby establishing a standardized reference baseline for future research.

The main contributions of this paper are as follows:

- We establish BacktestBench, a large-scale benchmark for automated quantitative backtesting. Built on over 6 million real market records, it covers metrics calculation, ticker selection, strategy selection, and parameter confirmation.
- We propose AutoBacktest, a robust multi-agent baseline system that translates natural language strategies into executable backtests via semantic factor extraction, SQL generation, and Python code execution.
- We evaluate 23 mainstream LLMs on BacktestBench and perform ablation studies to identify critical factors influencing end-to-end backtesting performance.

2 Dataset Construction

2.1 Data Source

To construct a backtesting dataset tailored to real-world scenarios, we perform rigorous cleaning and integration of raw stock data provided by Elmleaf Information Technology. First, we collect daily trading data for all stocks listed on the Shanghai, Shenzhen, and Beijing Stock Exchanges from January 2, 2020, to September 30, 2025. Post-cleaning, the dataset comprises 6,549,254 records covering 5,401 listed companies over a duration of 1,395 trading days. More detailed schema descriptions and additional statistics are deferred to Appendix C.1.

2.2 Core Concepts

In Quantitative Investing, we distinguish four closely related concepts: factor, signal, strategy, and KPI.

A *factor* is a numerical variable computed from raw market observations (e.g., open, high, low, close, volume) via a deterministic statistical or technical formula, and it quantifies a specific market pattern or state [33]. For example, the 5-day moving average (MA5) maps a price series to a scalar time series that represents the average price over the past five trading days.

A *signal* is a Boolean decision derived by applying a constraint or threshold rule to one or more factors at time t , indicating whether a trading condition is satisfied. For instance, a buy signal can be defined as $\text{Open}_t > \text{MA5}_{t-1}$, where the factor value uses only information available before time t to avoid look-ahead bias. Similarly, a sell signal can be defined by an opposite inequality or a different constraint on another factor.

A *strategy* specifies how to combine buy and sell signals into executable trading actions under a backtesting protocol, including the entry and exit logic, position state transitions, and required constraints (e.g., long-only, alternating buy and sell). For example, a simple long-only strategy enters when the buy signal fires, exits when the sell signal fires, and holds the position otherwise.

A *KPI* (key performance indicator) is a quantitative metric used to evaluate factors, signals, and strategies under a backtesting protocol, covering predictability, risk, and implementability.

In summary, factors provide the numeric basis, signals convert factors into actionable conditions, strategies compose buy and sell

signals into a complete and reproducible trading policy for backtesting, and KPIs provide the evaluation criteria that connect these concepts to measurable and comparable outcomes.

2.3 Factor and KPI Selection

2.3.1 Factor Selection. Facing a massive number of candidate factors, this study adheres to two core screening principles:

- **Data Availability:** The existing database schema must fully support the foundational fields required for factor calculation.
- **Computational Determinism:** Factors must possess strict mathematical formulations to ensure the uniqueness and reproducibility of calculation results. Based on this principle, we exclude factors involving randomness or subjective judgment, such as news sentiment scores generated by LLMs based on non-deterministic seeds, or soft metrics relying on manual annotation.

Based on these criteria, we select 43 factors spanning Risk, Quality, Momentum and Technical categories, including 4 sell-only, 2 buy-only and 37 dual-use factors. A detailed list of all factors is provided in Appendix A.

2.3.2 KPI Selection. Guided by domain experts, we select seven KPIs as the primary evaluation criteria: Return Ratio, Maximum Drawdown, Volatility, Annual Sharpe Ratio, Win Rate, Profit Loss Ratio, and Calmar Ratio [8]. The Calmar Ratio is defined as the ratio of annualized return to maximum drawdown and serves as a downside-risk-sensitive indicator. Detailed definitions of all seven KPIs are provided in Appendix Table 6.

2.4 Function Implementation

2.4.1 Implementation of Atomic Strategy Functions from Factors. To transform 43 theoretical factors into executable engineering objects, we encapsulate them into 80 atomic strategy functions using Pandas and NumPy. Each function couples factor computation with explicit signal-triggering logic, with 41 dedicated to sell-side and 39 to buy-side operations. These collectively form our signal pool.

Each atomic strategy function takes the target ticker’s historical data and specific hyperparameters (e.g., lookback window, decision threshold) as inputs. These functions encapsulate both factor computation and signal evaluation, returning a list of trading dates where the strategy is triggered. The *threshold* parameter is specifically used to define the boundary conditions for the logic, such as determining whether a value deviation is significant enough to generate a signal. To guarantee realistic backtesting, we strictly avoid look-ahead bias by ensuring that the decision to trade at day T relies exclusively on statistics computed from information available up to $T - 1$. An illustrative example is provided in the appendix A.

2.4.2 Implementation of KPI Computation Functions. Because several financial KPIs are sensitive to implementation choices and LLMs can hallucinate when reasoning about these computations, we adopt a standardized backtesting protocol with deterministic trading rules and unambiguous metric definitions to ensure unique and reproducible results.

The protocol fixes the execution microstructure to a long-only setup with strictly alternating buy and sell operations, disallows intraday round trips ($T+0$), executes buys at the opening price and sells at the closing price, and forcibly liquidates any remaining position at the end of the backtest window.

We simulate a fixed-capital portfolio without leverage, enforce round-lot trading (at least 100 shares), and set transaction costs and taxes to zero in the baseline experiments. We evaluate performance under a daily mark-to-market accounting convention with annualization based on 252 trading days per year and a constant daily risk-free rate of 0.0001 for the Annual Sharpe Ratio, and we assume the input time series is complete and strictly ordered by trading date. We provide the full specification of the execution mechanism, position sizing constraints, return computation, and data integrity assumptions in Appendix B.

The backtesting logic described above is implemented in Python and encapsulated into a dedicated metrics calculation function. This function takes as input a DataFrame containing market data together with the series of buy and sell signals generated by the signal pool, and returns the seven KPI values defined in the evaluation framework.

To ensure both correctness and robustness of the implementation, this study applies a double-blind verification procedure. Two quantitative engineering experts independently implement the same metric computation function strictly following the shared backtesting protocol, and then conduct cross-validation on ten randomly selected market data samples. Only when the outputs of the two implementations match exactly across all test cases is the implementation accepted as correct; the preferred version is then chosen as the final backtesting engine after discussion. This procedure effectively mitigates potential logical errors at the code level and enhances the reliability of the experimental evaluation toolkit.

2.5 Strategy Code Construction

To construct a diverse corpus for quantitative investing, we design an automated framework that dynamically synthesizes executable backtesting code. The framework operates by sampling and combining atomic strategy functions. Combinatorial analysis demonstrates that selecting up to four atomic functions without replacement yields a vast search space comprising 92,170 buy strategies and 112,791 sell strategies. Leveraging Python reflection, the system instantiates these atomic operators and maps them into four canonical decision tasks: metrics calculation, ticker selection, parameter confirmation, and strategy selection. Detailed construction procedures and generation statistics are provided in Appendix C.2.

Metrics Calculation. This task family evaluates the model’s ability to compute quantitative performance indicators for a single strategy under a fixed market environment. Each instance presents a natural language description of the strategy and a specified KPI (e.g., Sharpe Ratio or Maximum Drawdown), and the model is required to output the corresponding numerical value.

Ticker Selection. Ticker selection tasks emulate choosing the best-performing asset from a candidate universe under a shared strategy specification. Given a fixed strategy and multiple ticker histories, the model must identify the asset that optimizes the target KPI.

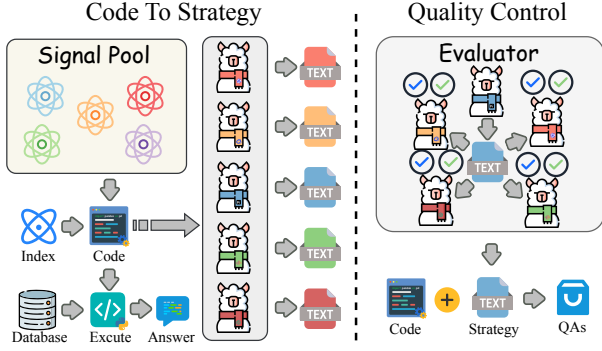


Figure 2: Pipeline of natural language strategy generation and evaluation.

Parameter Confirmation. Parameter confirmation tasks focus on selecting hyperparameters within an otherwise fixed strategy architecture. The model is asked to determine which candidate parameter value (such as a threshold or lookback window) yields the best KPI when applied to the same underlying asset and backtest window.

Strategy Selection. Strategy_selection tasks sit at the top of the decision hierarchy and require choosing among multiple competing strategy logics under identical market conditions. Given several candidate strategies sharing the same underlying, horizon, initial capital, and evaluation metric, the model must decide which logic achieves the highest performance.

Remarks. Across all four task families, each instance is paired with executable Python code, a standardized backtesting environment, and ground-truth labels, enabling end-to-end evaluation from natural language to quantitative outcomes. Further implementation details, including data sampling rules, code synthesis pipelines, and filtering criteria, are provided in Appendix C.2.

2.6 Strategy Description Generation

After constructing the corpus of executable strategy code, we adopt a code-to-text reverse-engineering paradigm to obtain human-readable strategy descriptions aligned with actual backtesting behavior. To ensure both semantic accuracy and financial soundness, we employ a multi-model generation and evaluation pipeline with strict acceptance rules.

2.6.1 Multi-Model Parallel Generation. We select five state-of-the-art open-source LLMs as generation agents: Kimi-K2-Thinking-BF16 [1], MiniMax-M2.1-BF16 [2], GLM-4.7 [36], GPT-OSS-120B-BF16 [18], and Qwen3-235B-A22B-Thinking-2507 [31]. These models are deployed on-premise using 72 NVIDIA A800-SXM4-80GB GPUs. For each strategy code sample, each of the five models independently performs reverse parsing and rewriting to produce diverse natural-language descriptions in an instructional style tailored to backtesting scenarios.

Crucially, the input to these models extends beyond raw function bodies. As illustrated in our codebase, each atomic strategy operator and KPI function is annotated with comprehensive docstrings

that explicitly define the *Strategy Logic*, *Mathematical Formula*, and *Input/Output Specifications*. This rich semantic context provides ground-truth guidance, significantly reducing ambiguity and minimizing the risk of model hallucination during the generation process. As illustrated on the left side of Figure 2, this procedure follows a code-to-strategy pipeline that transforms executable backtesting programs into aligned natural language strategies. Prompt design details, including how we enforce temporal consistency and separate system-level defaults from strategy-specific logic, are provided in Appendix C.3.

2.6.2 Automated Filtering. To ensure high corpus quality, we implement a rigorous automated filtering mechanism based on peer review. For a natural language strategy description generated by a specific model (e.g., model A), the other four models (e.g., models B, C, D, and E) serve as independent auditors. Each auditor evaluates the code-text pair along two dimensions: *code fidelity* and *strategy validity*. Code fidelity ensures the text faithfully reconstructs the program logic, while strategy validity verifies that the trading rules are financially logical and practically executable. We enforce a strict acceptance protocol: a sample is retained if and only if it receives unanimous consensus from all four peer evaluators across both dimensions (totaling eight positive votes). If multiple descriptions for a single strategy code pass this check, one is randomly selected to prevent duplication.

2.7 Dataset Statistics and Human Validation

Applying the rigorous filtering pipeline to the initial pool of 33,003 strategy programs yields a finalized corpus of 18,246 high-quality code-text pairs. These instances are stratified by task type into training (10,215), validation (4,195), and testing (3,836) sets. Detailed distributions regarding task families and comparative examples of accepted versus rejected descriptions are provided in Appendix C.3.

To statistically quantify the quality of the generated corpus, we conduct a human audit on a random sample of 200 instances (representing approximately 5% of the test set). Five volunteers assess the *completeness* of the strategy descriptions, reporting a pass rate of 97.5% (195/200). Simultaneously, six experts in Python implementation verify *code correctness*, confirming a 98.5% (197/200) consistency between the code and the trading rules.

Furthermore, to benchmark the *realism* of the synthetic data against human standards, we manually construct a separate set of 70 strategies across seven KPI categories. This expert-crafted subset is developed through a forward generation process that encompasses natural language description, SQL data retrieval, and backtesting implementation, which is subsequently subjected to rigorous expert cross-validation.

3 AutoBacktest

To address the challenges of automated backtesting posed by this dataset, we design a multi-agent framework that mimics the workflow of a quantitative researcher. Three specialized agents cooperate in a pipeline: the Summarizer parses natural language strategies into structured indicator representations, the Retriever generates and validates executable SQL to fetch historical market data, and the Coder produces and runs backtesting code to obtain final numerical results. A high-level description is given below, while interaction

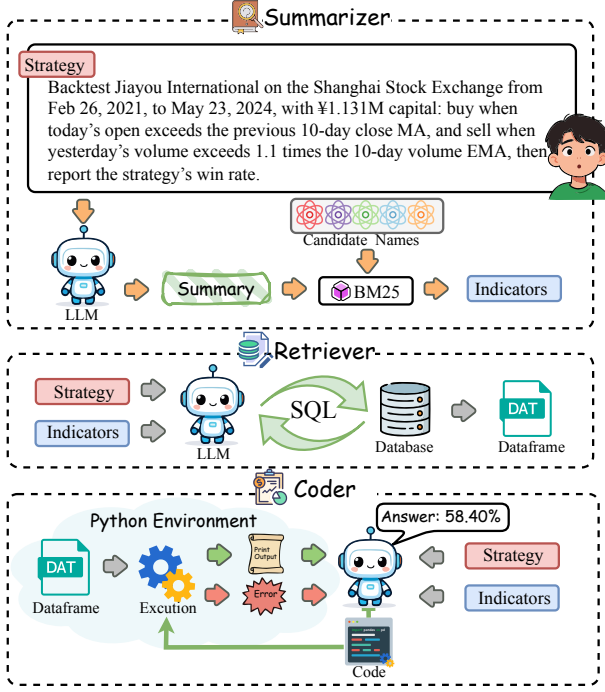


Figure 3: Overall framework of the AutoBacktest.

details, prompts, and implementation code are deferred to Appendix D.

3.1 Summarizer

The Summarizer first employs an LLM, guided by predefined prompts, to extract keywords related to factors and KPIs from the user’s natural language strategy. These summarized keywords then query a BM25 retrieval mechanism, which matches them against a comprehensive library to identify precise standard indicator names. The final output is a structured list of these verified standard indicator names, stored in the shared intermediate state. The concrete prompt design, JSON schema, and reference implementation are provided in Appendix D.

3.2 Retriever

The Retriever builds on the Summarizer’s output to construct the data layer for backtesting. It first maps the identified indicator names to their unique Short Codes—compact, token-efficient identifiers (e.g., mapping “13-day Bull Power” to `DELAY(HIGH, 1)–DELAY(EMA(CLOSE, 13), 1)`) that serve as unambiguous keys in the database schema. By injecting this enriched context (Name + Short Code) alongside the original strategy text, it prompts an LLM to generate a single executable SQL query. The agent then performs an execution-based validation loop to ensure that the statement runs successfully on the PostgreSQL database and returns a non-empty result. The finalized SQL string and its corresponding model message are preserved in the shared storage; further engineering details, including the exact SQL prompting pattern and error-handling logic, are described in Appendix D.

3.3 Coder

The Coder serves as the execution endpoint that transforms the retrieved data into final backtesting results. Its input consists of three key elements: the user’s natural language strategy, the mapped indicator context (names and Short Codes), and a data preview (head and tail rows) of the DataFrame obtained by the Retriever. Guided by a system prompt that enforces strict backtesting protocols, the Coder invokes a Python execution tool where the full DataFrame has been pre-injected into the runtime environment. The LLM then iteratively writes and debugs the code to calculate the required metrics or execute the strategy logic. Full implementation details and the answer extraction procedure are documented in Appendix D.

4 Experiments

4.1 Experimental Setup.

4.1.1 Model Configuration. Our experimental benchmark encompasses 23 LLMs, categorized into open-source and closed-source families. The open-source cohort spans several major families, including the Qwen3 series (ranging from 4B to the 235B A22B Mixture-of-Experts), the Ministral reasoning series (14B and 8B), the Kimi family (K2-Thinking and Linear 48B), the GLM family (4.7 and Flash), and the GPT-OSS family (120B and 20B), alongside standalone models such as MiniMax-M2.1, Seed OSS 36B, MIMO V2 Flash [29], and DeepSeek V3.2 [6]. The closed-source cohort comprises four leading proprietary models: Gemini 3 Pro, Qwen3 Max, Qwen3 Coder Plus, and Seed 1.8. See Appendix F.1 for detailed deployment specifications. Notably, while most models leverage CoT reasoning, Kimi Linear 48B and Seed OSS 36B operate without CoT. To ensure fair comparability and balance generation diversity with stability, we uniformly set the temperature parameter to 0.6 across all inference tasks.

4.1.2 Evaluation Metrics. Regarding evaluation metrics, this study quantifies model performance across three dimensions. First, for the factor retrieval task, we employ Accuracy, Precision, Recall, and F1 Score for comprehensive assessment. Second, for the SQL generation task, in addition to computing the Executable Rate (ECR) of generated SQL statements, we also focus on Execution Accuracy (EA) [34], which is defined as the degree of match between the result set returned by the generated SQL query and the ground-truth result set, ignoring element ordering. Finally, for the four categories of strategy backtesting problems, we report the accuracy of the final execution results produced by the model-generated strategy code, reflecting the end-to-end holistic performance. For metrics calculation problems, a prediction is counted as correct only if the absolute error between the predicted value and the ground-truth value is below 10^{-3} ; otherwise, it is treated as incorrect.

4.2 Main Results

4.2.1 Dominance of Closed-Source Models and the Catch-up of Open-Source Models. Table 1 reveals that the closed-source model Gemini 3 Pro [10] holds a dominant position in overall performance, achieving an Overall Accuracy (OA) of 67.41%. Notably, in the Metrics Calculation (MC) task, which demands the highest level of logical reasoning, it secures a top score of 51.67%. In contrast, while the best-performing open-source model GLM 4.7 achieves an

Table 1: This table reports model performance on the synthetic BacktestBench dataset and excludes results from the expert-crafted subset. Overall performance comparison across four task categories. OA denotes the overall performance aggregated across these four categories. Abbreviations: MC: Metrics Calculation, TS: Ticker Selection, PC: Parameter Confirmation, SS: Strategy Selection, ECR: Execution Correctness Rate, EA: Execution Accuracy.

Model	Back Test Accuracy					SQL Generation		Indicator Retrieval			
	OA	MC	TS	PC	SS	ECR	EA	Acc	P	R	F1
Open-Source Large Language Models											
Qwen3 235B	<u>55.01</u>	<u>34.71</u>	82.26	<u>75.93</u>	85.04	100.00	97.11	<u>76.41</u>	<u>93.62</u>	94.75	<u>94.18</u>
Qwen3 Next 80B	44.06	26.02	67.40	65.86	68.17	99.92	94.79	73.80	92.57	94.54	93.54
Qwen3 32B	47.81	27.93	75.93	70.34	72.21	100.00	90.72	67.41	90.44	93.99	92.18
Qwen3 30B	47.03	28.67	71.25	65.86	75.06	99.97	92.86	69.73	91.78	93.25	92.51
Qwen3 14B	36.97	16.96	62.04	61.57	64.61	99.95	87.59	60.92	87.56	90.50	89.01
Qwen3 8B	26.36	4.93	53.92	50.56	57.48	99.97	86.89	60.61	87.33	90.66	88.96
Qwen3 4B	19.63	1.77	39.06	42.54	48.22	98.85	69.94	58.34	87.16	89.84	88.48
Seed OSS 36B	11.34	4.60	20.08	13.06	28.50	91.16	87.96	68.07	90.49	91.89	91.18
Ministral 3 14B	26.98	7.76	47.73	50.93	58.91	97.18	72.26	59.62	86.77	90.75	88.71
Ministral 3 8B	17.26	2.83	34.11	37.31	36.34	92.23	46.35	47.99	82.20	87.69	84.86
Kimi K2 Thinking	44.97	30.02	66.57	58.02	67.46	98.20	96.06	79.82	95.09	96.86	95.96
Kimi Linear 48B	4.87	0.19	10.18	8.58	14.96	99.22	48.46	74.37	79.38	82.05	80.69
GLM 4.7	56.83	39.13	80.33	77.24	80.76	99.92	94.86	74.35	93.48	94.53	94.00
GLM 4.7 Flash	20.26	4.41	47.59	37.31	32.30	97.81	56.80	56.18	88.03	90.80	89.39
MiniMax M2.1	45.91	21.98	<u>81.71</u>	69.22	76.72	99.56	95.02	57.09	86.65	90.76	88.66
GPT OSS 120B	49.56	30.86	74.00	70.34	76.48	99.43	<u>96.58</u>	67.65	90.46	93.35	91.88
GPT OSS 20B	33.13	17.33	51.17	49.44	62.00	94.34	90.56	67.54	90.86	92.76	91.80
Mimo V2 Flash	40.43	17.43	73.04	64.37	71.26	97.84	80.42	58.58	86.67	93.38	89.90
Deepseek V3.2	50.70	29.18	79.78	73.51	<u>81.47</u>	99.97	92.86	68.98	91.01	<u>94.88</u>	92.90
Closed-Source Large Language Models											
Gemini 3 Pro	67.41	51.67	90.37	82.46	89.07	99.64	98.70	87.15	96.08	96.07	96.08
Qwen3 Max	61.84	44.14	85.56	82.28	85.27	100	94.73	72.42	91.75	95.82	93.74
Qwen3 Coder Plus	42.05	20.59	63.41	71.08	77.91	99.95	94.03	65.09	89.94	94.24	92.04
Seed 1.8	60.58	43.22	85.83	77.05	84.80	99.40	97.45	76.93	94.27	96.24	95.25

OA of 56.83% and an MC score of 39.13%, it lags behind Gemini 3 Pro by 10.58 percentage points in OA and 12.54 percentage points in MC. This gap indicates that open-source models continue to face significant bottlenecks in complex financial logic operations.

4.2.2 Sensitivity Differences to Scaling Laws Across Tasks. Data analysis based on the Qwen3 series (ranging from 4B to 235B parameters) in Table 1 uncovers significant differences in sensitivity to model parameter size across different tasks. Logical reasoning tasks are highly sensitive to parameter scale: when the model size is reduced from 235B to 4B, the Metrics Calculation (MC) score plummets from 34.71% to 1.77%, resulting in an almost complete loss of computational capability. Conversely, indicator retrieval tasks exhibit lower sensitivity to parameter scale: in the same comparison group, the F1 score for Indicator Retrieval only declined from 94.18% to 88.48%, with Qwen3 4B maintaining a relatively high retrieval standard. This suggests that while small-parameter

models struggle to complete complex strategy logic calculations, they still possess good potential for basic information retrieval.

4.2.3 Decoupling of Syntax and Logic Capabilities Due to Lack of CoT. The evaluation of non-CoT models in Table 1 highlights a significant decoupling between syntactic proficiency and logical reasoning. Kimi Linear 48B exemplifies this with a high Execution Correctness Rate (ECR) of 99.22% but a low Execution Accuracy (EA) of 48.46% and a minimal Metrics Calculation (MC) score of 0.19%, indicating the generation of executable yet logically flawed code. Similarly, Seed OSS 36B achieves a strong EA of 87.96% but a low Overall Accuracy (OA) of 11.34%, demonstrating that successful data retrieval fails to guarantee downstream reasoning success. These results confirm that CoT is essential for integrating code generation with the rigorous logical deduction necessary for quantitative backtesting.

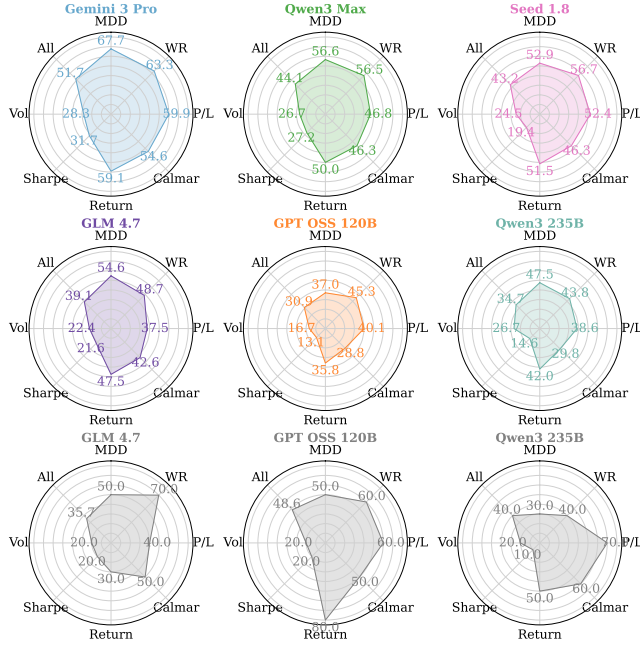


Figure 4: Detailed model performance on the Metrics Calculation task. All denotes the overall performance on the entire Metrics Calculation task. Abbreviations: Sharpe: Annual Sharpe Ratio, Calmar: Calmar Ratio, MDD: Maximum Drawdown, P/L: Profit Loss Ratio, Return: Return Ratio, Vol: Volatility, WR: Win Rate. The first two rows summarize results on the synthetic BacktestBench dataset, while the grey radar chart in the last row reports performance on the expert-crafted evaluation set.

4.2.4 Accuracy Analysis of Metrics Calculation Tasks. For metrics calculation problems involving seven types of KPIs, we visualized the performance of the top 3 open-source and top 3 closed-source models (see Figure 4).

Complex Mathematical Logic is the Biggest Bottleneck. Figure 4 highlights a steep decline in model performance as KPI complexity increases. While logically simpler metrics like Win Rate and Maximum Drawdown see higher accuracy, complex statistical indicators such as Volatility and Sharpe Ratio remain “disaster zones” across all models, with significantly lower pass rates. This trend confirms that as tasks escalate from basic arithmetic to intricate statistical operations (e.g., variance, annualization), the reliability of LLM-generated code decays precipitously (see Appendix E.2 for detailed error analysis).

Closed-Source Models’ Moat on “High-Order Logic”. In Figure 4, Gemini 3 Pro leads with a comprehensive accuracy (All) of 51.67%, with its core advantage lying in its mastery of high-difficulty indicators. On the most error-prone Sharpe and Vol indicators, Gemini 3 Pro maintains a significant lead (approximately 4.48 and 1.62 percentage points ahead of the second-place Qwen3 Max, respectively). This implies that top-tier closed-source models possess stronger logical rigor when understanding complex financial mathematical

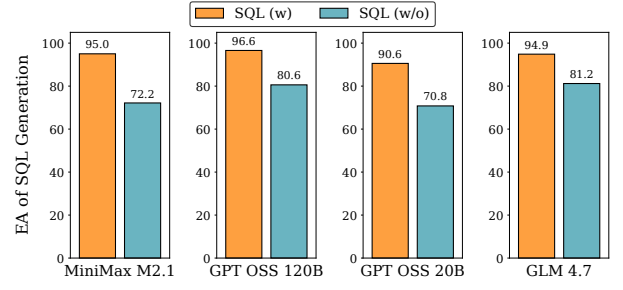


Figure 5: Short Code Ablation.

formulas and translating them into precise Python/SQL implementations, capable of handling details such as annualization coefficient adjustments and division-by-zero protection.

Open-Source Models: Strong in Extremes, Weak in Statistics. GLM 4.7, representing open-source models (All 39.13%), achieved an accuracy of 54.55% on MDD (Maximum Drawdown) calculation, a score that even surpasses the closed-source model Seed 1.8 (52.86%). The core logic of MDD is “tracking the maximum decline from historical net value peaks,” which is a typical “extreme value tracking” logic. However, on the Vol KPI involving “distribution statistics” logic, GLM 4.7 only scored 22.37%. This contrast illustrates that top open-source models are already perfectly capable of generating procedural calculation code with clear logic but still exhibit high error rates (e.g., formula misuse or library function call errors) when dealing with abstract statistical calculations.

“Logic Collapse” Zone of Base Models. Taking GPT OSS 120B as an example, its scores on Vol and Sharpe KPIs were only 16.71% and 13.06%, respectively. This indicates that some models almost completely lose correctness in code implementation when facing complex financial calculations. Such low scores likely stem from the models’ inability to correctly understand the conversion relationship between “annualized volatility” and “daily volatility,” or ignoring data preprocessing (such as logarithmic return transformation) during coding, resulting in calculation results that deviate significantly from standard values.

Synthetic Data Faithfully Mirrors Expert-crafted Subset Difficulty. Beyond absolute scores, comparing the synthetic and expert-crafted subset radar plots in Figure 4 reveals that their outer contours are highly aligned: models consistently perform worst on Vol and Sharpe, while achieving much higher accuracy on simpler KPIs. This shape-level consistency indicates that the synthetic BacktestBench data faithfully preserves the intrinsic difficulty hierarchy of real-world backtest metrics, suggesting that both datasets share a similar underlying task distribution rather than reflecting artifacts of our data generation pipeline.

4.3 Ablation Study

4.3.1 Impact of Short Code on SQL Generation. We analyze the impact of the Short Code mechanism by comparing SQL generation performance with and without its inclusion. As shown in Figure 5, the introduction of Short Codes consistently enhances Execution

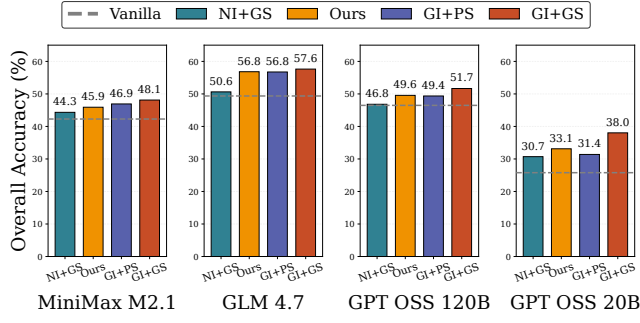


Figure 6: Ground Truth Ablation.

Accuracy (EA) across all evaluated models. Specifically, MiniMax M2.1 and GPT OSS 20B exhibit the most pronounced performance jumps, with EA improvements exceeding 20 percentage points, while robust performers like GPT OSS 120B and GLM 4.7 also achieve clear gains. These results confirm that Short Codes act as critical semantic anchors, effectively guiding the models to map natural language intents to precise database schema elements.

4.3.2 Independent Impact of Short Code and SQL Quality. To deeply investigate the independent effects of Short Code and SQL generation quality on strategy backtesting performance, we designed five experimental configurations for comparative analysis. **Vanilla** serves as the baseline, where the LLM generates SQL and backtesting code directly from the user query without any Short Code information. **Ours** represents the complete implementation of AutoBacktest, including retrieval-augmented Short Code context. Additionally, we introduce three variants: **GI+PS** (Gold Indicator + Predicted SQL) uses the ground-truth Short Code (Gold Indicator) as context but lets the LLM predict the SQL; **NI+GS** (No Indicator + Gold SQL) uses no Short Code but provides the ground-truth SQL (Gold SQL); and **GI+GS** (Gold Indicator + Gold SQL) provides both ground-truth Short Code and Gold SQL, serving as the theoretical performance upper bound.

The experimental results in Figure 6 reveal the decisive role of Short Code in the strategy generation pipeline. Overall, all configurations incorporating Short Code information (**Ours**, **GI+PS**, **GI+GS**) significantly outperform those without it (**Vanilla**, **NI+GS**). Remarkably, although the **NI+GS** method uses perfect Gold SQL, its performance, while better than the Vanilla baseline, still lags substantially behind the three groups containing Short Code. This phenomenon strongly demonstrates that including Short Code in the prompt significantly enhances the LLM’s ability to understand and apply financial indicators, yielding performance gains far exceeding those from optimizing SQL statements alone. Furthermore, comparing **Ours** with **GI+PS** reveals minimal performance differences, with **Ours** even slightly outperforming in some GPT OSS models (likely due to random experimental error). This further indicates that given accurate Short Code context, the LLM can generate correct final backtesting code through contextual understanding even if the generated SQL has minor imperfections, highlighting the robustness of the AutoBacktest framework against SQL errors.

5 Related Work

5.1 LLM-based Strategy Backtesting

Recent works have explored LLMs for quantitative tasks, yet gaps remain in backtesting rigor. Automate Strategy Finding [12] and QuantAgent [28] focus on factor mining and self-improving trading agents, prioritizing profitability over the standardization and reproducibility of the underlying code execution. Similarly, FinMem [35] addresses long-term memory for trading decisions but overlooks the challenges of complex data retrieval and metrics calculation. While AutoPrep [9] automates general tabular data preprocessing, it lacks the specialized temporal logic required to prevent look-ahead bias in financial backtesting.

5.2 Datasets for Strategy Backtesting

Existing benchmarks primarily target Reinforcement Learning or specific sub-tasks rather than the full backtesting workflow. TradeMaster [26] and FinRL-Meta [15] provide extensive market environments for training RL agents but do not evaluate the generation of interpretable strategy logic. FNSPID [7] aligns news with market data but serves mainly as an information retrieval resource. While QuantEval [11] assesses strategy coding, it remains limited to a small scale of 60 problems. Similarly, StockBench [4] prioritizes the profitability of agentic trading decisions, and Market-Bench [24] evaluates the implementation accuracy of introductory strategies. In contrast, BacktestBench pioneers a large-scale evaluation of the end-to-end quantitative research workflow. It spans natural language understanding, SQL retrieval, and rigorous backtest execution, comprehensively assessing the translation of complex natural language into executable backtesting logic.

6 Conclusion

In this paper, we introduce BacktestBench, a pioneering benchmark dedicated to automated quantitative strategy backtesting, a domain characterized by complex temporal logic and rigorous precision requirements. To support this initiative, we construct a large-scale dataset with 18,246 high-quality QA pairs derived from real-world market records, covering four core decision-making tasks: metrics calculation, ticker selection, strategy selection, and parameter confirmation. Complementing this resource, we propose AutoBacktest, a multi-agent collaboration framework that mimics the professional workflow of quantitative researchers to achieve end-to-end automation from natural language strategy descriptions to executable backtesting code. We systematically evaluate 23 mainstream LLMs on BacktestBench and conduct comprehensive ablation studies to identify key performance drivers in this domain. By establishing a standardized evaluation protocol and a rigorous data resource, this work bridges the gap between general-purpose code generation and quantitative investment research, laying a solid foundation for future advancements in intelligent financial decision-making.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62272050 and the grant of Beijing Normal- Hong Kong Baptist University sponsored by

Guangdong Provincial Department of Education; in part by Zhuhai Science-Tech Innovation Bureau under Grant No. 2320004002772 and the Interdisciplinary Intelligence Super Computer Center of Beijing Normal University (Zhuhai).

References

- [1] Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, ZhaoWei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, and Haiming Wang. 2025. Kimi K2: Open Agentic Intelligence. *CoRR* abs/2507.20534 (2025). arXiv:2507.20534 doi:10.48550/ARXIV.2507.20534
- [2] Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, Chengjun Xiao, Chengyu Du, Chi Zhang, Chu Qiao, Chunhao Zhang, Chunhui Du, Congchao Guo, Da Chen, Deming Ding, Dianjun Sun, Dong Li, Enwei Jiao, Haigang Zhou, Haimo Zhang, Han Ding, Haohai Sun, Haoyu Feng, Huaiguang Cai, Haichao Zhu, Jian Sun, Jiaqi Zhuang, Jiaren Cai, Jiayuan Song, Jin Zou, Jingyang Li, Jinhao Tian, Jinli Liu, Junhao Xu, Junjie Yan, Junteng Liu, Junxian He, Kaiyi Feng, Ke Yang, Kecheng Xiao, Le Han, Leyang Wang, Lianfei Yu, Liheng Feng, Lin Li, Lin Zheng, Ling Du, Lingyu Yang, Lunbin Zeng, Minghui Yu, Mingliang Tao, Mingyuan Chi, Mozhi Zhang, Mujie Lin, Nan Hu, Nongyu Di, Peng Gao, Pengfei Li, Pengyu Zhao, Qibing Ren, Qidi Xu, Qile Li, Qin Wang, Rong Tian, Ruitao Leng, Shaoxiang Chen, Shaoyu Chen, Shengmin Shi, Shitong Weng, Shuchang Guan, Shuai Yu, Sichen Li, Songquan Zhu, Tengfei Li, Tianchi Cai, Tianrun Liang, Weiyu Cheng, Weize Kong, Wenkai Li, Xiancai Chen, Xiangjun Song, Xiao Luo, Xiao Su, Xiaobo Li, Xiaodong Han, Xinzhu Hou, Xuan Lu, Xun Zou, Xuyang Shen, Yan Gong, Yan Ma, Wang, Yiqi Shi, Yiran Zhong, and Yonghong Duan. 2025. MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. *CoRR* abs/2506.13585 (2025). arXiv:2506.13585 doi:10.48550/ARXIV.2506.13585
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [4] Yanxu Chen, Zijun Yao, Yantao Liu, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. 2025. StockBench: Can LLM Agents Trade Stocks Profitably In Real-world Markets? *CoRR* abs/2510.02209 (2025). arXiv:2510.02209 doi:10.48550/ARXIV.2510.02209
- [5] Marcos Lopez De Prado. 2018. *Advances in financial machine learning*. John Wiley & Sons.
- [6] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *CoRR* abs/2512.02556 (2025). arXiv:2512.02556 doi:10.48550/ARXIV.2512.02556
- [7] Zihan Dong, Xinyu Fan, and Zhiyuan Peng. 2024. FNSPID: A Comprehensive Financial News Dataset in Time Series. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 4918–4927. doi:10.1145/3637528.3671629
- [8] Martin Eling and Frank Schuhmacher. 2007. Does the choice of performance measure influence the evaluation of hedge funds? *Journal of Banking & Finance* 31, 9 (2007), 2632–2647.
- [9] Meihao Fan, Ju Fan, Nan Tang, Lei Cao, Guoliang Li, and Xiaoyong Du. 2025. AutoPrep: Natural Language Question-Aware Data Preparation with a Multi-Agent Framework. *Proc. VLDB Endow.* 18, 10 (2025), 3504–3517. doi:10.14778/3748191.3748211
- [10] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, et al. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] <https://arxiv.org/abs/2503.19786>
- [11] Zhaolu Kang, Junhao Gong, Wenqing Hu, Shuo Yin, Kehan Jiang, Zhicheng Fang, Yingjie He, Chunlei Meng, Rong Fu, Dongyang Chen, Leqi Zheng, Eric Hanchen Jiang, Yunfei Feng, Yitong Leng, Junfan Zhu, Xiaoyou Chen, Xi Yang, and Richeng Xuan. 2026. QuantEval: A Benchmark for Financial Quantitative Tasks in Large Language Models. arXiv:2601.08689 [cs.CL] <https://arxiv.org/abs/2601.08689>
- [12] Zhizhuo Kou, Holam Yu, Jingshu Peng, and Lei Chen. 2024. Automate Strategy Finding with LLM in Quant investment. *CoRR* abs/2409.06289 (2024). arXiv:2409.06289 doi:10.48550/ARXIV.2409.06289
- [13] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=XmProj9cPs>
- [14] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 1835, 28 pages.
- [15] Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. 2022. FinRL-meta: market environments and benchmarks for data-driven financial reinforcement learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 134, 15 pages.
- [16] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=UnUwSIgK5W>
- [17] Malik Magdon-Ismael and Amir F Atiya. 2004. Maximum drawdown. *Risk Magazine* 17, 10 (2004), 99–102.
- [18] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *CoRR* abs/2508.10925 (2025). arXiv:2508.10925 doi:10.48550/ARXIV.2508.10925
- [19] Robert Pardo. 2011. *The evaluation and optimization of trading strategies*. John Wiley & Sons.
- [20] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoning Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR* abs/2308.12950 (2023). arXiv:2308.12950 doi:10.48550/ARXIV.2308.12950
- [21] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: language models can teach themselves to use tools. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 2997, 13 pages.
- [22] William F Sharpe. 1998. The sharpe ratio. *Streetwise—the Best of the Journal of Portfolio Management* 3, 3 (1998), 169–85.
- [23] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 377, 19 pages.
- [24] Abhay Srivastava, Sam Jung, and Spencer Mateega. 2025. Market-Bench: Evaluating Large Language Models on Introductory Quantitative Trading and Market Dynamics. *CoRR* abs/2512.12264 (2025). arXiv:2512.12264 doi:10.48550/ARXIV.2512.12264
- [25] Michael Stonebraker, Lawrence A. Rowe, and Michael Hirohama. 2019. The implementation of POSTGRES. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, Michael L. Brodie (Ed.). ACM Books, Vol. 22. ACM / Morgan & Claypool, 519–559. doi:10.1145/3226595.3226639
- [26] Shuo Sun, Molei Qin, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An. 2023. TradeMaster: a holistic quantitative trading platform empowered by reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 2576, 15 pages.

- [27] Alan M. Turing. 1990. Computing Machinery and Intelligence. In *The Philosophy of Artificial Intelligence*, Margaret A. Boden (Ed.). Oxford University Press, 40–66.
- [28] Saizhuo Wang, Hang Yuan, Lionel M. Ni, and Jian Guo. 2024. QuantAgent: Seeking Holy Grail in Trading by Self-Improving Large Language Model. *CoRR* abs/2402.03755 (2024). arXiv:2402.03755 doi:10.48550/ARXIV.2402.03755
- [29] LLM-Core Xiaomi. 2026. MiMo-V2-Flash Technical Report. arXiv:2601.02780 [cs.CL] <https://arxiv.org/abs/2601.02780>
- [30] Qianqian Xie, Weiguang Han, Xiao Zhang, Yanzhao Lai, Min Peng, Alejandro Lopez-Lira, and Jimin Huang. 2023. PIXIU: a large language model, instruction data and evaluation benchmark for finance. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 1454, 16 pages.
- [31] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jian Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *CoRR* abs/2505.09388 (2025). arXiv:2505.09388 doi:10.48550/ARXIV.2505.09388
- [32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X
- [33] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. 2023. Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, New York, NY, USA, 5476–5486. doi:10.1145/3580305.3599831
- [34] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. doi:10.18653/v1/D18-1425
- [35] Yangyang Yu, Haoqiang Li, Zhi Chen, Yuechen Jiang, Yang Li, Jordan W. Suchow, Denghui Zhang, and Khaldoun Khashanah. 2025. FinMem: A Performance-Enhanced LLM Trading Agent With Layered Memory and Character Design. *IEEE Trans. Big Data* 11, 6 (2025), 3443–3459. doi:10.1109/TBDATA.2025.3593370
- [36] Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, Kedong Wang, Lucen Zhong, Mingdao Liu, Rui Lu, Shulin Cao, Xiaohan Zhang, Xuancheng Huang, Yao Wei, Yean Cheng, Yifan An, Yilin Niu, Yuanhao Wen, Yushi Bai, Zhengxiao Du, Zihan Wang, Zilin Zhu, Bohan Zhang, Bosi Wen, Bowen Wu, Bowen Xu, Can Huang, Casey Zhao, Changpeng Cai, Chao Yu, Chen Li, Chendi Ge, Chenghua Huang, Chenhui Zhang, Chenxi Xu, Chenzheng Zhu, Chuang Li, Congfeng Yin, Daoyan Lin, Dayong Yang, Dazhi Jiang, Ding Ai, Erle Zhu, Fei Wang, Gengzheng Pan, Guo Wang, Hailong Sun, Haitao Li, Haiyang Li, Haiyi Hu, Hanyu Zhang, Hao Peng, Hao Tai, Haoke Zhang, Haoran Wang, Haoyu Yang, He Liu, He Zhao, Hongwei Liu, Hongxi Yan, Huan Liu, Huilong Chen, Ji Li, Jiajing Zhao, Jiamin Ren, Jian Jiao, Jiani Zhao, Jianyang Yan, Jiaqi Wang, Jiayi Gui, Jiayue Zhao, Jie Liu, Jijie Li, Jing Li, Jing Lu, Jingsen Wang, Jingwei Yuan, Jingxuan Li, Jingzhao Du, Jinhua Du, Jinxin Liu, Junkai Zhi, Junli Gao, Ke Wang, Lekang Yang, Liang Xu, Lin Fan, Lindong Wu, Lintao Ding, Lu Wang, Man Zhang, Minghao Li, Minghuan Xu, Mingming Zhao, and Mingshu Zhai. 2025. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. *CoRR* abs/2508.06471 (2025). arXiv:2508.06471 doi:10.48550/ARXIV.2508.06471

A Factors

This section provides a comprehensive reference for all factors and KPIs used throughout BacktestBench. Table 5 presents the complete inventory of factors employed in the benchmark, while Tables 6 and 7 detail the correspondence between full names and short codes for factors and KPIs, respectively.

We additionally provide an atomic strategy function example in Figure 7. The example AmountMa6SellStrategy triggers a sell

signal when the trading volume on day $T - 1$ significantly exceeds the average volume of the preceding 6 days (from $T - 2$ to $T - 7$). Here, the *threshold* parameter serves as a multiplier to define what constitutes a significant volume spike; a signal is generated only if the volume at $T - 1$ is greater than the historical average multiplied by this threshold. This design strictly avoids look-ahead bias by comparing the observed volume at $T - 1$ against a baseline computed solely from data prior to $T - 1$, ensuring the decision at day T relies only on historically available information.

It is important to note that factors appearing similar but differing in time windows (e.g., 5-day vs. 20-day moving averages) are treated as distinct factors in our benchmark. This distinction is crucial because these variations capture fundamentally different market dynamics: shorter windows reflect short-term momentum and volatility patterns, while longer windows indicate long-term trends and structural movements. Consequently, each time-window variant represents a unique factor with distinct predictive characteristics and cannot be considered equivalent or interchangeable.

The factor and KPI naming system serves a critical role in the multi-stage AutoBacktest pipeline. During the Summarizer stage, the system retrieves and identifies indicators (include factors and KPIs) using their full names (e.g., “5-day Moving Average” or “Annual Sharpe Ratio”). Subsequently, in the Retriever and Coder stages, both the full names and their corresponding short codes are injected into the prompt context. This dual representation enables the LLM to understand the semantic meaning through full names while generating concise, standardized code using short codes, thereby ensuring consistency and reducing ambiguity in the automated backtesting process.

B Backtesting Protocol and Standardized Trading Rules

Several financial KPIs are sensitive to implementation details and can be mis-specified in non-standardized settings. Moreover, LLMs are prone to hallucinations when reasoning about these calculations. Therefore, we define a standardized backtesting and trading protocol that enforces deterministic rules and unambiguous computational logic, ensuring that the backtest results are unique and reproducible.

Execution Mechanism. The backtesting framework adopts a long-only trading scheme with strictly alternating buy and sell operations, and it explicitly disallows intraday round trips ($T+0$). All trades execute at observable and well-defined prices: buy orders fill at the opening price (Buy at Open) and sell orders fill at the closing price (Sell at Close), which removes ambiguity about intraday price paths. If a position remains open on the final day of the backtest window, the framework forcibly liquidates it at the corresponding closing price.

Allocation and Constraints. The simulation uses a fixed-capital setup with initial wealth denoted by $Cash_0$, and it does not allow leverage. On each buy date t , the trade size Q_t is computed dynamically as

$$Q_t = \left\lfloor \frac{Cash_t}{Price_{open,t}} \right\rfloor,$$

Table 2: Schema of the stocks database.

Column	Type
name	TEXT
stock_code	TEXT
trade_date	DATE
opening_price	NUMERIC
closing_price	NUMERIC
highest_price	NUMERIC
lowest_price	NUMERIC
volume_traded	NUMERIC
amount_traded	NUMERIC
percentage_change	NUMERIC

subject to a minimum order lot constraint $Q_t \in \mathbb{Z}_{\geq 100}$, which corresponds to trading in round lots of 100 shares. On sell dates, the framework enforces full liquidation of the current position. To isolate the impact of trading logic, the baseline experiments set transaction costs and taxes to zero.

Performance Evaluation. The framework adopts a daily mark-to-market convention when computing returns. The portfolio value PV_t at date t is the sum of the market value of all open positions marked at the closing price and the remaining cash balance. The daily return is computed as

$$r_t = \frac{PV_t - PV_{t-1}}{PV_{t-1}},$$

with $r_t = 0$ on days without an open position. Annualized return and annualized volatility follow the convention of 252 trading days per year, and the daily risk-free rate is fixed to 0.0001 when evaluating the Annual Sharpe Ratio. The Calmar Ratio is computed as the ratio of annualized return to maximum drawdown, which emphasizes downside-risk-adjusted performance.

Data Integrity Assumption. The backtest assumes that the input price time series is complete and strictly ordered by trading date. The current framework does not explicitly handle missing trading days, irregular calendars, or out-of-order records, and we leave these data quality issues for future extensions.

C Datasets

C.1 Database Schema

Table 2 presents the schema of the stocks PostgreSQL [25] database, detailing the 15 constituent columns along with their data types and descriptions. This unified schema is applied consistently across the following three exchange tables:

- beijing_stock_exchange
- shenzhen_stock_exchange
- shanghai_stock_exchange

C.2 Strategy Data Generation Details

This subsection details the construction pipeline for the four task families introduced in Section 3. Unless otherwise specified, all strategy instances are generated from the signal pool and KPI library described in Section A under the standardized backtesting protocol.

Table 3: Overall dataset statistics for BacktestBench across four core Quantitative Investing task families: metrics calculation, ticker selection, parameter confirmation, and strategy_selection.

Statistics	Num
# Total Questions	18,246
# Train	10,215
# Validation	4,195
# Test	3,836
# Metrics Calculation	10,244
# Ticker Selection	3,455
# Parameter Confirmation	2,548
# Strategy Selection	1,999
# Avg. Length per Strategy	141.7

Foundational Framework and Metrics Calculation Tasks. For metrics calculation tasks, the generation pipeline consists of three stages: signal pool sampling, logic composition, and backtest-based validation. First, the system samples a single stock from the three Chinese exchanges according to predefined weights and extracts a contiguous historical window of at least 252 trading days, which is standardized into a DataFrame containing OHLCV fields. Next, using code introspection, the framework independently samples up to four buy-side and four sell-side atomic factors, fuses their signals via set intersection (trades occur only when all selected factors fire on the same date), and randomly chooses one KPI as the optimization objective, assembling them into an executable Python strategy script. Finally, the script is run in an isolated interpreter, the KPI value is recorded as the ground-truth answer, and low-quality samples with sparse trades or anomalous outputs (e.g., NaN or infinite values) are filtered out, yielding a total of **17,082** metrics calculation programs.

Ticker Selection Tasks. Ticker selection tasks extend the above framework to a multi-asset setting. Instead of a single underlying, the system samples a candidate set of stocks, applies the same randomly generated strategy and KPI objective to each asset in parallel, and runs backtests under identical experimental conditions. According to the economic meaning of the KPI, the framework adopts either a maximization principle (for reward-oriented metrics such as return or Sharpe Ratio) or a minimization principle (for downside or risk metrics such as Maximum Drawdown or Volatility) to label the best-performing ticker as the ground truth, resulting in **6,098** ticker selection programs.

Parameter Confirmation Tasks. Parameter confirmation tasks emulate controlled hyperparameter tuning within a fixed strategy and environment. The system first constructs a baseline strategy with a deterministic factor combination and KPI, then selects one atomic factor (e.g., a moving-average breakout rule) as the optimization target and samples a discrete set of candidate parameter values from a predefined space. For each candidate, a strategy variant is formed by substituting the target parameter while keeping all other logic unchanged; all variants are backtested on the same price path and evaluated under the same KPI-based selection rule, and the

parameter yielding the best KPI is recorded as the label, producing **4,709** parameter confirmation programs.

Strategy Selection Tasks. Strategy selection tasks capture the decision problem of choosing among heterogeneous strategy logics under a shared market environment. The framework samples a small set of candidate strategies (e.g., a binary contest $\{A, B\}$ or ternary contest $\{A, B, C\}$), independently generates buy and sell factor combinations for each candidate, and constrains all of them to trade the same underlying over the same historical window with identical initial capital and KPI. Dynamic code generation then embeds all candidate strategies into a single script and runs parallel backtests; by comparing KPI values under the appropriate maximization or minimization criterion, the system identifies the best-performing logic (such as “B”) as the ground-truth label, yielding **5,114** strategy_selection programs.

SQL Query Construction and Annotation. For every strategy instance, the framework automatically constructs and annotates an SQL statement that exactly reproduces the underlying data slice used in backtesting. All queries follow a standard SELECT-FROM-WHERE pattern: during sampling, the chosen exchange, ticker, and date interval are mapped respectively to the FROM clause, ticker constraint, and time filter (e.g., FROM shenzhen_stock_exchange WHERE name IN (‘Ping An Bank’) AND trade_date BETWEEN ‘2019-01-15’ AND ‘2020-03-20’). After strategy code generation, the system parses the code to extract the actually used columns (such as opening_price, closing_price, volume_traded) and constructs a minimal SELECT clause containing only required fields, thereby avoiding redundant data loading. The finalized SQL string is stored together with the strategy code, backtest interval, and ticker list in a JSON file, ensuring a one-to-one correspondence between code, data, and environment that facilitates exact reproduction and error tracing.

C.3 Natural Language Generation and Evaluation Details

This subsection provides additional implementation details for the natural-language description generation and quality assessment pipeline introduced in Section 3.

Prompt Design and Generation Setup. We employ five specific on-premise models for generation: Kimi-K2-Thinking-BF16, MiniMax-M2.1-BF16, GLM-4.7, GPT-OSS-120B-BF16, and Qwen3-235B-A22B-Thinking-2507. All five models support CoT reasoning and are configured as instruction-following LLMs. The prompts enforce an instructional tone that mimics realistic backtesting requests, explicitly separate strategy-specific logic from system-level defaults (such as capital allocation and execution microstructure), and require precise temporal expressions (e.g., “based on $T - 1$ data”) to avoid look-ahead bias.

Evaluation Criteria and Reporting. During evaluation, each of the four evaluator models produces a structured JSON report for every code-text pair, containing binary decisions for code fidelity and strategy validity, together with brief error diagnoses (e.g., “parameter mismatch”, “future information usage”, or “missing exit rule”).

Only descriptions that receive positive judgments from all evaluators are retained, and the remaining reports are used to categorize and analyze common failure modes.

Scale and Resource Usage. The full pipeline processes 33,003 strategy programs and issues 825,075 LLM calls across the five models deployed on 72 NVIDIA A800-SXM4-80GB GPUs. Under the strict acceptance rule, the final corpus contains 10,244 metrics calculation tasks, 3,455 ticker selection tasks, 2,548 parameter confirmation tasks, and 1,999 strategy selection tasks, totaling 18,246 samples partitioned into 10,215 training, 4,195 validation, and 3,836 test instances. Additional dataset statistics are presented in Table 3.

D Agent Data Flow and Interaction

In the AutoBacktest framework, each strategy instance is represented as a shared record initialized with a natural-language description stored in the strategy field. The Summarizer, Retriever, and Coder agents operate sequentially on this record, progressively transforming the raw text into a fully specified backtesting sample containing factor definitions, executable data queries, and quantitative results.

The Summarizer is the semantic entry point of the system. Its input consists of the fields strategy, and optionally the global definition of the Financial Indicator vocabulary (which explicitly encompasses both the 43 trading factors and 7 performance KPIs). Based on this information, the Summarizer analyzes the strategy description, extracts the mentioned indicators, normalizes informal wording into canonical Financial Indicator names, and produces a structured list of predicted indicators together with any auxiliary annotations needed for later retrieval. This list is written back into the shared record as fields such as predict_indicators and a refined version predict_indicators(BM25), which serve as the semantic bridge from free-form language to the financial indicator library.

The Retriever builds on the enriched record produced by the Summarizer. Its input includes the original strategy text strategy, the predicted indicator list predict_indicators(BM25), and a separate financial indicator dictionary that maps each financial indicator name to its implementation-level identifiers, for example short codes or database column names. Using these inputs, the Retriever constructs a compact textual context that pairs financial indicators with their short codes and then generates a single SQL statement that specifies which ticker, time interval, and raw fields should be fetched from the historical database for this strategy. Before the statement is accepted, the Retriever executes it against the database to check for syntax correctness and non-empty results; if necessary, it iteratively revises the query based on error messages or empty outputs. Once a valid query is obtained, the final SQL string is stored in the shared record as predict_SQL.

The Coder is the execution endpoint that turns the verified data-access specification into a concrete backtesting result. Its input is the fully enriched record after the Retriever stage, including the strategy text strategy, the strategy type strategy_type (which determines whether the task is metrics calculation, ticker selection, parameter confirmation, or strategy selection), the indicator context predict_indicators(BM25) together with the indicator dictionary, and the validated SQL statement predict_SQL. The

Coder first executes `predict_SQL` to obtain a time series of market data corresponding to the requested ticker and period; this data is treated as the backtesting environment, and an empty result triggers an early termination for that record with an error flag. If a non-empty data slice is obtained, the Coder constructs a task-specific prompt that combines a preview of the retrieved table, the normalized indicator information, and the strategy description, and then uses a tool-augmented language model to write and execute backtesting code within this environment. The final model message is parsed to extract a standardized answer, such as a KPI value, an index of the best-performing ticker, a selected parameter, or a chosen strategy. The final model message is parsed to extract a standardized answer, such as a KPI value, an index of the best-performing ticker, a selected parameter, or a chosen strategy. The extracted scalar answer is appended to the shared record as the `pred_answer` field.

E Experiments

E.1 Impact of BM25 in Summarizer

To evaluate the necessity of the retrieval mechanism in our framework, we conduct an ablation study on the Summarizer module. Specifically, we compare the performance of various LLMs with and without the integration of the BM25 algorithm. In the configuration without BM25 (“w/o”), the Summarizer relies solely on the LLM’s internal knowledge to identify relevant intents and slots. Conversely, the configuration with BM25 (“w”) augments the prompt with retrieved context related to indicator definitions and project metadata.

Table 4 presents the comparative results. The integration of BM25 yields a significant performance improvement across all tested models. For instance, DeepSeek V3.2 exhibits a substantial increase in Accuracy from 17.65% to 68.98% and an F1 score improvement from 66.92% to 92.90%. Even larger models, such as Qwen3 235B, benefit markedly, with Accuracy rising from 30.42% to 76.41%. These results demonstrate that while LLMs possess strong reasoning capabilities, the precise identification of domain-specific indicators and intents in quantitative investing requires external knowledge retrieval. The BM25 algorithm effectively bridges this gap by providing accurate context, thereby significantly enhancing the Summarizer’s ability to map natural language queries to correct structured representations.

E.2 Case Study

To diagnose the root causes of performance degradation in *metrics calculation* tasks—specifically Volatility and Sharpe Ratio—we construct a targeted analysis framework for the *backtesting* process. In our experiments with Gemini-3-Pro, we filter for samples with accurate SQL queries and successful *indicator* retrieval, isolating instances where Python code execution produces unexpected deviations. Our analysis identifies two primary mechanisms driving these failures.

E.2.1 Case 1: Composite Attribution Analysis of Volatility Calculation Failure. This case examines the failure mechanism in volatility calculation tasks, where three systemic logical defects in the generated code cumulatively distort *backtesting* results.

The primary error occurs during performance attribution, where the code fails to properly initialize the return series. By neglecting

to insert the initial capital as a “Day 0” baseline, the first-day return is calculated as invalid and subsequently discarded. This left-side truncation compromises the integrity of the time series, forcing the volatility calculation to rely on an incomplete sample set and introducing significant numerical errors.

Furthermore, regarding *indicator* construction, the model misinterprets the mathematical definition of “Mean Absolute Deviation (MAD)” within the Commodity Channel Index (CCI). Instead of the standard “average absolute deviation within a rolling window,” the code implements a “simple rolling average of absolute price deviations.” This algorithmic discrepancy causes buy signal triggers to diverge from the intended strategy.

Additionally, the trading logic lacks necessary boundary constraints for the *backtesting* cycle. Without a filter to prohibit position opening on the final day, the system executes non-compliant intraday bidirectional trading (opening at market start and forced liquidation at close). This artificially inflates transaction costs and skews the final net value, rendering the volatility calculation ineffective.

E.2.2 Case 2: Systematic Bias in Sharpe Ratio Assessment. This case investigates the failure mechanism affecting the Sharpe Ratio, a metric whose accuracy relies heavily on the completeness of the return series and strict adherence to trading logic. We observe that logical defects in two key dimensions cause severe distortion in the assessment.

A critical flaw appears in the construction of the net value sequence, where the code omits the “zero-point anchoring” operation. Failing to explicitly insert the initial capital at the head of the sequence leads to the exclusion of the first-day return during difference calculations. This reduction in sample size affects both the numerator (annualized excess return) and the denominator (return volatility), resulting in a Sharpe Ratio derived from a fragmented time window and destroying statistical validity.

Simultaneously, the code exhibits insufficient boundary control at the end of the *backtesting* period. By permitting non-compliant “open and close on the same day” operations, the system incurs invalid transaction costs that are directly borne by the net value sequence. This artificially depresses the final return performance, depriving the generated Sharpe Ratio of its reference value as a performance benchmark.

F Impletation Details

F.1 LLM Deployments

This section details the LLM configurations employed across the different stages of our research. The deployment is divided into two distinct phases: the data construction phase and the experimental evaluation phase.

During the data construction phase, we utilized a high-performance computing cluster equipped with 72 NVIDIA A800-SXM4-80GB GPUs to deploy five state-of-the-art open-source CoT models. These models were selected for their superior reasoning capabilities and include Kimi-K2-Thinking-BF16, MiniMax-M2.1-BF16, GLM-4.7, GPT-OSS-120B-BF16, and Qwen3-235B-A22B-Thinking-2507. Detailed specifications for these models are provided in Table 8. To optimize inference efficiency, we leveraged SGLang and vLLM

Table 4: Performance Comparison of the Summarizer Module With and Without BM25 Retrieval. This table evaluates the effectiveness of incorporating the BM25 algorithm into the Summarizer stage across various LLMs. “w/o” denotes the baseline performance without BM25, while “w” indicates performance with BM25 integration. The metrics include Accuracy (Acc), Precision (P), Recall (R), and F1 Score.

Model	BM25 Status	Acc (%)	P (%)	R (%)	F1 (%)
DeepSeek V3.2	w/o	17.65	64.35	69.71	66.92
	w	68.98	91.01	94.88	92.90
Qwen3 4B	w/o	13.03	61.24	63.06	59.52
	w	58.34	87.16	89.84	88.48
Qwen3 8B	w/o	15.64	60.98	65.24	63.04
	w	60.61	87.33	90.66	88.96
Qwen3 14B	w/o	11.47	57.59	59.52	55.79
	w	60.92	87.56	90.50	89.01
Qwen3 30B	w/o	20.36	68.10	70.04	69.06
	w	69.73	91.78	93.25	92.51
Qwen3 Next 80B	w/o	27.42	73.88	76.39	75.12
	w	73.80	92.57	94.54	93.54
Qwen3 235B	w/o	30.42	75.55	77.19	76.36
	w	76.41	93.62	94.75	94.18
GLM 4.7	w/o	24.35	72.06	73.78	72.91
	w	74.35	93.48	94.53	94.00
MiniMax M2.1	w/o	14.21	60.19	65.33	62.65
	w	57.09	86.65	90.76	88.66

as our deployment frameworks. A unified temperature parameter of 0.6 was applied across all models to maintain consistency in generation diversity and stability.

In the experimental phase, we expanded our infrastructure to 100 NVIDIA A800-SXM4-80GB GPUs to locally host 18 models for comprehensive benchmarking. To complement these local deployments, we accessed several proprietary models via external APIs. Specifically, Mimo V2 Flash and Gemini 3 Pro were accessed through OpenRouter, incurring a total cost of \$756. For DeepSeek V3.2, Qwen3 Max, and Qwen3 Coder Plus, we utilized their official compatible-mode APIs available at <https://dashscope.aliyuncs.com/compatible-mode/v1>. Additionally, Seed 1.8 was accessed via the Volcengine API at <https://ark.cn-beijing.volces.com/api/v3>. To ensure a fair comparison between local and API-based models, we strictly maintained the temperature setting at 0.6 for all inferences throughout the experiment.

F.2 Agent Impletation

We implement the AutoBacktest agent using the LangGraph framework, employing a ReAct (Reasoning and Acting) architecture to orchestrate the backtesting workflow. The agent interacts with a PostgreSQL database to retrieve financial data, which is converted into a Pandas DataFrame. We equip the agent with a Python REPL tool (`create_python_repl_tool`), enabling it to execute Python code directly on the dataframe to perform complex calculations and logic verification.

To ensure robustness and standardized outputs, the implementation features several key components:

- (1) **Dynamic Context Construction:** The system dynamically generates prompts by injecting the specific trading strategy, indicator definitions retrieved via the BM25 Summarizer, and the dataframe schema. This ensures the LLM possesses all necessary context for code generation.
- (2) **Structured Output Parsing:** We define specific Pydantic models (e.g., `ClassA_FinalAnswerFormat`) for different task types, such as metric calculation or ticker selection. This enforces strict output formatting, facilitating accurate answer extraction from the agent’s response.
- (3) **Execution Constraints:** To balance reasoning depth with efficiency and prevent infinite loops, the agent is configured with a maximum step limit of 25 iterations per query.

The system also includes a robust answer extraction mechanism that handles various response formats, including CoT traces (e.g., `<think>` tags) and JSON structures, ensuring reliable evaluation of the model’s predictions.

G Prompts

This section presents the five critical prompts utilized in our work, categorized into two distinct phases: Dataset Construction and the AutoBacktest Agent Workflow.

G.1 Dataset Construction Prompts

The construction of the BacktestBench dataset relies on a “Code-to-Text” reverse engineering paradigm. Figure 9 displays the **Code-to-Strategy Prompt**, which is used to translate synthetically generated atomic strategy code into natural language descriptions. This prompt instructs the LLM to interpret the Python logic and describe the trading rules in a tone mimicking a human quantitative researcher. Subsequently, to ensure the high quality and logical consistency of these generated descriptions, we employ the **Strategy Evaluation Prompt** shown in Figure 10. This prompt guides a separate set of LLMs to cross-validate the generated text against the original code, checking for discrepancies in logic, parameter values, or potential hallucinations.

G.2 AutoBacktest Agent Prompts

The inference phase involves three specialized agents, each driven by a carefully designed prompt to handle specific sub-tasks in the quantitative workflow.

- **Summarizer Prompt** (Figure 11): This prompt guides the Summarizer agent to analyze the user’s raw natural language query. Its primary function is to identify and extract the *full names* of relevant financial indicators (e.g., “5-day Moving Average”) and Key Performance Indicators (KPIs). It serves as the semantic parser that maps unstructured text to the standardized terminology of our indicator library.
- **Retriever Prompt** (Figure 12): Once the indicators are identified, the Retriever agent uses this prompt to locate the necessary data. Crucially, at this stage, the system injects both the *full names* and their corresponding *Short Codes* (e.g., SMA(CLOSE, 5)) into the prompt context. This enables the LLM to precisely understand which database fields are required and to construct accurate SQL queries for fetching the underlying market data.
- **Coder Prompt** (Figure 13): Finally, the Coder agent utilizes this prompt to generate the executable Python backtesting script. Similar to the Retriever, this prompt is enriched with the indicator *Short Codes*. By explicitly providing these standardized short codes, we constrain the LLM to use our pre-defined, rigorously tested calculation logic rather than hallucinating arbitrary formulas. This ensures that the generated code is not only syntactically correct but also financially valid and aligned with the user’s original intent.

H Limitations

While this study establishes a rigorous benchmark for evaluating LLMs in the domain of quantitative finance and demonstrates the potential of “code-to-text” reverse engineering to align natural language instructions with executable trading logic, several limitations remain in the current data scope and task design.

First, the dataset primarily focuses on short-term timing strategies driven by daily price fluctuations. It does not yet incorporate long-term value investing strategies that require synthesizing multi-modal information, such as macroeconomic indicators, corporate financial reports, or unstructured news text.

Second, the research objective is strictly defined as assessing the capabilities of LLMs within the “strategy understanding–data

retrieval–code generation–backtesting verification” pipeline. Consequently, the model outputs are optimized for feasibility verification and code correctness rather than for generating alpha-generating investment advice for live deployment.

Third, constrained by a database that currently supports only daily market data, all backtesting simulations adhere to a simplified daily mark-to-market logic (“buy at open, sell at close”). This approach does not account for real-world execution frictions such as slippage, intraday price path dependence, or high-frequency microstructure details.

Furthermore, to isolate and rigorously test the model’s tool-use and code-implementation abilities, the experimental setup abstracts away certain complex market constraints. The framework does not explicitly model dynamic transaction fees, varying tax rates, or regime-switching minimum trading units, nor does it address advanced portfolio management tasks like dynamic position sizing or mean-variance optimization. Methodologically, the benchmark is built around the classical “indicator–signal–backtest” paradigm; it does not currently cover strategies based on machine learning models (e.g., time series forecasting or reinforcement learning) or complex meta-strategies involving indicator weighting and ensemble methods, primarily due to the prohibitive computational costs of grid-searching such high-dimensional spaces.

In summary, the present benchmark serves as a foundational baseline for testing LLMs’ strategy understanding and code generation in a structured financial environment. However, significant room remains for future expansion into more realistic settings that model multi-modal inputs, multi-frequency data, and dynamic market constraints.

Table 5: All factors used in strategy construction. Type indicates whether the factor serves as a buy signal, sell signal, or both.

Factor	Factor full name	Type
Amount MA 6	6-day MA of Trading Amount	both
Amount MA 20	20-day MA of trading amount	both
Amount Std 6	6-day Standard Deviation of Trading Amount	sell
ATR 6	6-day Average True Range	both
ATR 20	20-day Average True Range	both
BBI	Bull and Bear Index	both
BIAS 5	5-day Bias	both
BIAS 60	60-day Bias	both
Bollinger Lower Band	Bollinger Lower Band	buy
Bollinger Upper Band	Bollinger Upper Band	sell
Bull Power 13	13-day Bull Power	buy
CCI 15	15-day Commodity Channel Index	both
CCI 20	20-day Commodity Channel Index	both
EMA 5	5-day Exponential Moving Average	both
EMA 12	12-day Exponential Moving Average	both
MA 5	5-day Moving Average	both
MA 10	10-day Moving Average	both
MA 20	20-day Moving Average	both
MA 60	60-day Moving Average	both
MACD	Moving Average Convergence Divergence	both
Net Money Flow 20	20-day Net Money Flow	both
PLRC 12	12-day Price Linear Regression Coefficient	both
Price MA Ratio 5	Price to 5-day MA Ratio	both
Price MA Ratio 20	Price to 20-day MA Ratio	both
Price MA Ratio 120	Price to 120-day MA Ratio	both
PSY 6	6-day Psychological Line	both
PSY 12	12-day Psychological Line	both
Return Kurtosis 60	60-day Kurtosis of Daily Returns	sell
Return Skewness 60	60-day Skewness of Daily Returns	both
Return Variance 20	20-day Variance of Daily Returns	sell
ROC 6	6-day Rate of Change	both
ROC 12	12-day Rate of Change	both
ROC 20	20-day Rate of Change	both
ROC 120	120-day Rate of Change	both
RSI	Relative Strength Index	both
Sharpe Ratio 120	120-day Sharpe ratio	both
Single Day VPT	Single-day Price Volume Trend	both
Volume EMA 5	5-day Exponential Moving Average of Volume	both
Volume EMA 10	10-day EMA of Volume	both
Volume EMA 26	26-day Exponential Moving Average of Volume	both
Volume MA 12	12-day Moving Average of Volume	both
Volume ROC 12	12-day Volume Rate of Change	both
Volume Std 10	10-day Standard Deviation of Trading Volume	both

Table 6: Factors with Short Code.

Factor	Short Code
Amount MA 6	DELAY(AMOUNT,1)/DELAY(SMA(AMOUNT,6),2)
Amount MA 20	DELAY(AMOUNT,1)/DELAY(SMA(AMOUNT,20),2)
Amount Std 6	STD(DELAY(AMOUNT,1),6)/SMA(STD(DELAY(AMOUNT,1),6),20)
ATR 6	DELAY(EMA(MAX(MAX(HIGH-LOW,ABS(HIGH-DELAY(CLOSE,1))),ABS(LOW-DELAY(CLOSE,1))),6),1)
ATR 20	DELAY(EMA(MAX(MAX(HIGH-LOW,MAX(ABS(HIGH-DELAY(CLOSE,1))),ABS(LOW-DELAY(CLOSE,1)))),20),1)
BBI	(SMA(DELAY(CLOSE,1),3)+SMA(DELAY(CLOSE,1),6)+SMA(DELAY(CLOSE,1),12)+SMA(DELAY(CLOSE,1),24))/4
BIAS 5	(DELAY(CLOSE,1)-SMA(DELAY(CLOSE,1),5))/SMA(DELAY(CLOSE,1),5)*100
BIAS 60	DELAY((CLOSE-SMA(CLOSE,60))/SMA(CLOSE,60)*100,1)
Bollinger Lower Band	SMA(DELAY(CLOSE,1),20)-2.0*STD(DELAY(CLOSE,1),20)
Bollinger Upper Band	SMA(DELAY(CLOSE,1),20)+2.0*STD(DELAY(CLOSE,1),20)
Bull Power 13	DELAY(HIGH,1)-DELAY(EMA(CLOSE,13),1)
CCI 15	DELAY(((HIGH+LOW+CLOSE)/3-SMA((HIGH+LOW+CLOSE)/3,15))/((0.015*SMA(ABS((HIGH+LOW+CLOSE)/3-SMA((HIGH+LOW+CLOSE)/3,15)),15)),1)
CCI 20	DELAY(((HIGH+LOW+CLOSE)/3-SMA((HIGH+LOW+CLOSE)/3,20))/((0.015*SMA(ABS((HIGH+LOW+CLOSE)/3-SMA((HIGH+LOW+CLOSE)/3,20)),20)+1e-20),1)
EMA 5	DELAY(EMA(CLOSE,5),1)
EMA 12	DELAY(EMA(CLOSE,12),1)
MA 5	SMA(DELAY(CLOSE,1),5)
MA 10	SMA(DELAY(CLOSE,1),10)
MA 20	SMA(DELAY(CLOSE,1),20)
MA 60	SMA(DELAY(CLOSE,1),60)
MACD	(EMA(DELAY(CLOSE,1),12)-EMA(DELAY(CLOSE,1),26))-EMA(EMA(DELAY(CLOSE,1),12)-EMA(DELAY(CLOSE,1),26),9)
Net Money Flow 20	DELAY(SUM(SIGN((HIGH+LOW+CLOSE)/3-DELAY((HIGH+LOW+CLOSE)/3,1))*(HIGH+LOW+CLOSE)/3*VOLUME,20),1)
PLRC 12	LINEARREG_SLOPE(DELAY(CLOSE,1),12)
Price MA Ratio 5	OPEN/SMA(DELAY(CLOSE,1),5)
Price MA Ratio 20	OPEN/SMA(DELAY(CLOSE,1),20)
Price MA Ratio 120	OPEN/SMA(DELAY(CLOSE,1),120)
PSY 6	(100/6)*SUM(DELAY(IF(CLOSE>DELAY(CLOSE,1),1,0),1),6)
PSY 12	(100/12)*SUM(IF(DELAY(CLOSE,1)>DELAY(CLOSE,2),1,0),12)
Return Kurtosis 60	DELAY(SMA(((CLOSE/DELAY(CLOSE,1)-1)-SMA((CLOSE/DELAY(CLOSE,1)-1),60))*4,60)/SMA(((CLOSE/DELAY(CLOSE,1)-1)-SMA((CLOSE/DELAY(CLOSE,1)-1),60))*2,60)*2-3,1)
Return Skewness 60	DELAY(SKEW(PERCENTAGE_CHANGE, 60), 1)
Return Variance 20	DELAY(VAR(CLOSE/DELAY(CLOSE,1)-1,20),1)
ROC 6	(DELAY(CLOSE,1)-DELAY(CLOSE,7))/DELAY(CLOSE,7)*100
ROC 12	(DELAY(CLOSE,1)-DELAY(CLOSE,13))/DELAY(CLOSE,13)*100
ROC 20	(DELAY(CLOSE,1)-DELAY(CLOSE,21))/DELAY(CLOSE,21)*100
ROC 120	(DELAY(CLOSE,1)-DELAY(CLOSE,121))/DELAY(CLOSE,121)*100
RSI	DELAY(100-100/(1+EMA(MAX(CLOSE-DELAY(CLOSE,1),0),14)/EMA(ABS(MIN(CLOSE-DELAY(CLOSE,1),0)),14)),1)
Sharpe Ratio 120	DELAY((SMA((CLOSE/DELAY(CLOSE,1)-1),120)-0.0001)*SQRT(252)/STD((CLOSE/DELAY(CLOSE,1)-1),120),1)
Single Day VPT	(DELAY((CLOSE/DELAY(CLOSE,1)-1)*VOLUME,1)-SMA(DELAY((CLOSE/DELAY(CLOSE,1)-1)*VOLUME,2),20))/STD(DELAY((CLOSE/DELAY(CLOSE,1)-1)*VOLUME,2),20)
Volume EMA 5	DELAY(VOLUME,1)/DELAY(EMA(VOLUME,5),1)
Volume EMA 10	DELAY(VOLUME/EMA(VOLUME,10),1)
Volume EMA 26	DELAY(VOLUME,1)/DELAY(EMA(VOLUME,26),1)
Volume MA 12	DELAY(VOLUME,1)>DELAY(SMA(VOLUME,12),1)
Volume ROC 12	(DELAY(VOLUME,1)-DELAY(VOLUME,13))/DELAY(VOLUME,13)*100
Volume Std 10	STD(DELAY(VOLUME,1),10)/SMA(STD(DELAY(VOLUME,1),10),20)

Table 7: KPIs with Calculation Codes. (Note: PV denotes Portfolio Value; PNL denotes Profit and Loss; N represents the calculation window.)

KPI	KPI full name	Short Code
Return	Return Ratio	$(\text{CASH_FINAL} + \text{POSITION_FINAL} * \text{CLOSE}) / \text{INITIAL_CAPITAL} - 1$
MDD	Max Drawdown	$\text{MAX}((\text{CUMMAX}(\text{CASH} + \text{POSITION} * \text{CLOSE}) - (\text{CASH} + \text{POSITION} * \text{CLOSE})) / \text{CUMMAX}(\text{CASH} + \text{POSITION} * \text{CLOSE}))$
Vol	Volatility	$\text{STD}(\text{PV} / \text{DELAY}(\text{PV}, 1) - 1, N)$
Sharpe	Annual Sharpe Ratio	$(\text{MEAN}((\text{PV} - \text{DELAY}(\text{PV}, 1)) / \text{DELAY}(\text{PV}, 1)) - 0.0001) / \text{STD}((\text{PV} - \text{DELAY}(\text{PV}, 1)) / \text{DELAY}(\text{PV}, 1)) * \text{SQRT}(252)$
WR	Win Rate	$\text{SUM}(\text{IF}(\text{PNL} > 0, 1, 0)) / \text{COUNT}(\text{PNL}) * 100$
P/L	Profit Loss Ratio	$(\text{SUM}(\text{IF}(\text{PNL} > 0, \text{PNL}, 0)) / \text{ABS}(\text{SUM}(\text{IF}(\text{PNL} < 0, \text{PNL}, 0)))) * (\text{SUM}(\text{IF}(\text{PNL} < 0, 1, 0)) / \text{SUM}(\text{IF}(\text{PNL} > 0, 1, 0)))$
Calmar	Calmar Ratio	$((\text{POW}(\text{LAST}(\text{PV}) / \text{FIRST}(\text{PV}), 252 / \text{COUNT}(\text{PV})) - 1) / \text{ABS}(\text{MIN}((\text{PV} - \text{CUMMAX}(\text{PV})) / \text{CUMMAX}(\text{PV}))))$

Table 8: Comparison of Model Inference Configurations. Abbreviations: Backend = Inference Backend, MFS = mem-fraction-static, CoT = Chain of Thought, Algo. = Speculative Algorithm, Steps = speculative-num-steps, TopK = speculative-eagle-topk, Draft = speculative-num-draft-tokens.

Model	GPUs	Backend	MFS	CoT	Algo.	Steps	TopK	Draft
Qwen3-235B-A22B-Thinking-2507 ¹	8	SGLang	0.8	Yes	EAGLE3 ²	3	1	4
Qwen3 Next 80B A3B Thinking ³	4	SGLang	0.8	Yes	NEXTN	5	8	32
Qwen3 32B ⁴	4	SGLang	0.8	Yes	None	-	-	-
Qwen3 30B A3B Thinking 2507 ⁵	2	SGLang	0.9	Yes	None	-	-	-
Qwen3 14B ⁶	1	SGLang	0.9	Yes	None	-	-	-
Qwen3 8B ⁷	1	SGLang	0.9	Yes	None	-	-	-
Qwen3 4B ⁸	1	SGLang	0.9	Yes	None	-	-	-
Seed OSS 36B ⁹	2	vLLM	0.9	No	None	-	-	-
Ministral 3 14B Reasoning 2512 ¹⁰	2	vLLM	0.9	Yes	None	-	-	-
Ministral 3 8B Reasoning 2512 ¹¹	1	vLLM	0.9	Yes	None	-	-	-
Kimi-K2-Thinking-BF16 ¹²	32	SGLang	0.9	Yes	None	-	-	-
Kimi Linear 48B A3B Instruct ¹³	4	SGLang	0.8	No	None	-	-	-
GLM-4.7 ¹⁴	16	SGLang	0.8	Yes	EAGLE ¹⁵	3	1	4
GLM 4.7 Flash ¹⁶	4	vLLM	0.8	Yes	None	-	-	-
MiniMax-M2.1-BF16 ¹⁷	8	SGLang	0.9	Yes	None	-	-	-
GPT-OSS-120B-BF16 ¹⁸	8	SGLang	0.8	Yes	EAGLE3 ¹⁹	3	1	4
GPT OSS 20B BF16 ²⁰	2	SGLang	0.9	Yes	None	-	-	-

¹<https://www.modelscope.cn/models/Qwen/Qwen3-235B-A22B-Thinking-2507>²<https://www.modelscope.cn/models/lmsys/Qwen3-235B-A22B-EAGLE3>³<https://www.modelscope.cn/models/Qwen/Qwen3-Next-80B-A3B-Thinking>⁴<https://www.modelscope.cn/models/Qwen/Qwen3-32B>⁵<https://www.modelscope.cn/models/Qwen/Qwen3-30B-A3B-Thinking-2507>⁶<https://www.modelscope.cn/models/Qwen/Qwen3-14B>⁷<https://www.modelscope.cn/models/Qwen/Qwen3-8B>⁸<https://www.modelscope.cn/models/Qwen/Qwen3-4B>⁹<https://huggingface.co/ByteDance-Seed/Seed-OSS-36B-Instruct>¹⁰<https://huggingface.co/mistralai/Ministral-3-14B-Reasoning-2512>¹¹<https://huggingface.co/mistralai/Ministral-3-8B-Reasoning-2512>¹²<https://www.modelscope.cn/models/unsloth/Kimi-K2-Thinking-BF16>¹³<https://www.modelscope.cn/models/moonshotai/Kimi-Linear-48B-A3B-Instruct>¹⁴<https://www.modelscope.cn/models/ZipuAI/GLM-4.7>¹⁵https://docs.sglang.io/basic_usage/glm45.html¹⁶<https://www.modelscope.cn/models/ZipuAI/GLM-4.7-Flash>¹⁷<https://www.modelscope.cn/models/QuixiAI/MiniMax-M2.1-bf16>¹⁸<https://www.modelscope.cn/models/unsloth/gpt-oss-120b-BF16>¹⁹<https://www.modelscope.cn/models/nv-community/gpt-oss-120b-Eagle3>²⁰<https://www.modelscope.cn/models/unsloth/gpt-oss-20b-BF16>

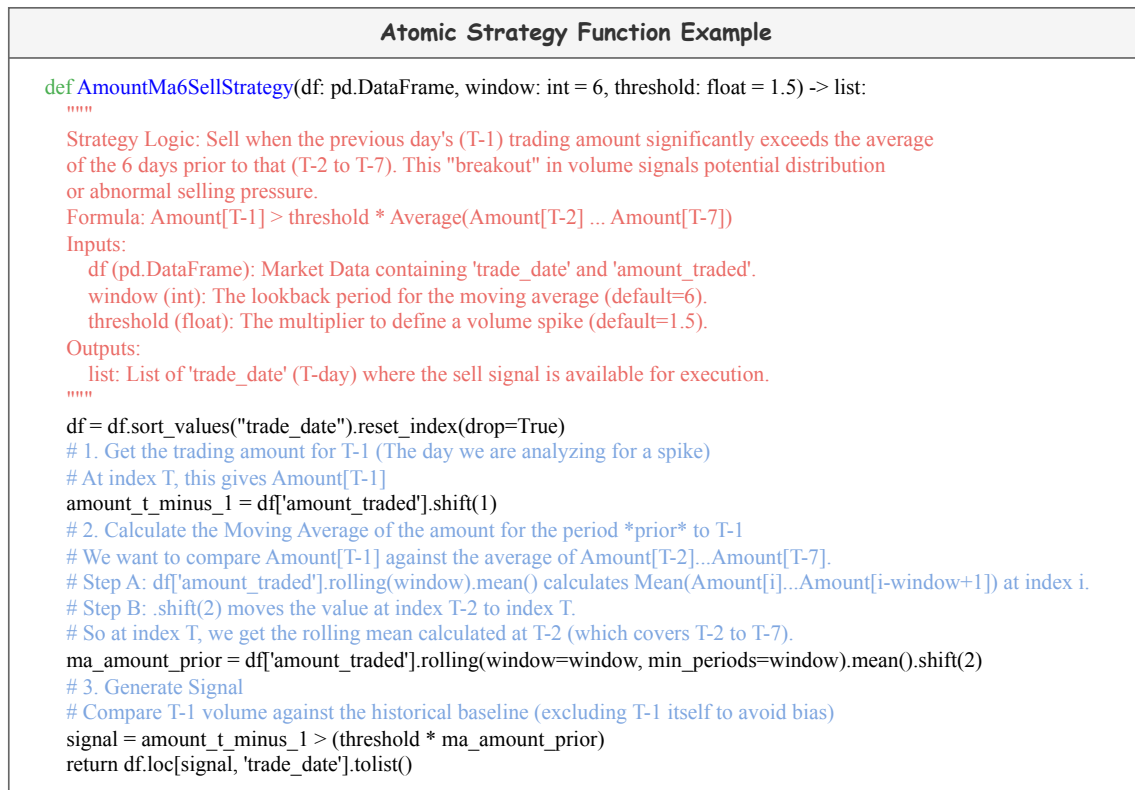


Figure 7: Atomic Strategy Function Example.

QA Example

strategy: We want to backtest China Nuclear Engineering Corporation Limited on the Shanghai Stock Exchange from 29 Nov 2022 to 13 Mar 2025 with an initial capital of CNY 1,970,000. Please open a position whenever yesterday's traded volume is greater than the 12-day moving average of volume calculated from the 12 days ending on yesterday (i.e., including yesterday). Close the entire position when yesterday's traded amount exceeds 1.5 times the 20-day moving average of amount computed from the 20 days ending the day before yesterday (i.e., excluding yesterday). Finally, compute the Calmar ratio for this strategy.

strategy_type: "metrics_calculation"

factors: ["20-day Moving Average of Trading Amount", "12-day Moving Average of Volume", "Calmar Ratio"]

KPI: "Calmar Ratio"

SQL_statement: SELECT amount_traded, closing_price, opening_price, trade_date, volume_traded FROM shanghai_stock_exchange WHERE name IN ('China Nuclear Engineering Corporation Limited') AND trade_date >= '2022-11-29' AND trade_date <= '2025-03-13';

code: """

```
import pandas as pd
import numpy as np
def VolumeMA12BuyStrategy(df: pd.DataFrame, window: int = 12) -> list:
    .....
    return df.loc[condition, 'trade_date'].tolist()
def AmountMa20SellStrategy(df: pd.DataFrame, window: int = 20, threshold: float = 1.5) -> list:
    .....
    return df.loc[sigal, 'trade_date'].tolist()
def calculate_strategy_calmar_ratio(df, buy_dates, sell_dates, initial_capital):
    .....
    return round(calmar_ratio, 6)
# ===== Execution Logic Section =====
# Stock Information
initial_capital = 1970000 # Initial Capital
stock_name = ['China Nuclear Engineering Corporation Limited'] # Stock Name
result = DATAFRAME
# Calculate buy/sell signals
buy_dates = list(set(VolumeMA12BuyStrategy(result)))
sell_dates = list(set(AmountMa20SellStrategy(result)))
# Calculate Strategy Performance Metric
strategy_target = calculate_strategy_calmar_ratio(result, buy_dates, sell_dates, initial_capital)
"""
```

answer: 0.41591

Figure 8: QA example.

Prompt for Converting Python Code to Natural Language Strategy

You are a quantitative researcher submitting a backtest request. Your task is to translate the Python code logic into a direct, natural language request for a simulation.

1. Goal: Request Narrative

- Tone: Use an Instructional/Requesting tone (e.g., "We want to backtest...", "Please execute...").
- Precision: While the tone should be natural, the logic must be mathematically precise. Do not sacrifice accuracy for brevity.
- Flow: Write as a continuous, logical request paragraph.

2. Implicit Framework Rules (STRICTLY OMIT THESE)

The following are system defaults and must NOT be mentioned. Focus ONLY on the unique strategy logic:

1. Order Sizing: Do not mention minimum 100 shares, odd lots, integers, or full position exits.
2. Session Logic: Do not mention "No buying on last day" or "Forced close on last day."
3. Conflict Resolution: Do not mention simultaneous signal priority.
4. Capital Management: Do not mention fixed capital limits or cash sufficiency checks.
5. Execution: Do not mention "Buy at Open / Sell at Close" unless the code *deviates* from this standard.
6. Math Details: Do not explain standard return formulas, annualization, or window warmups.
7. Data Handling: Do not mention NaN handling.

3. Language Guidelines

- Humanize Time (BUT BE PRECISE):

- Forbidden: "T", "T-1", "Day t".
- Required: "Today," "yesterday," "the previous day," "the past 20 days."

- Time Reference Precision (CRITICAL):

- Never describe an indicator vaguely (e.g., "Check the RSI").
- Always specify the reference day (e.g., "Check yesterday's RSI," "Calculate the MA using data prior to today").
- If a calculation uses 'shift(1)', explicitly state it is based on yesterday's data.

- Standardize Terms:

- Forbidden: Variable names ('rolling_mean', 'ma_5').
- Required: Financial terms ("Moving Average," "Volume Volatility").
- Clean Numbers: Write "0" instead of "0.0".

4. Required Content

Embed the following into your request naturally:

1. Scope: Exchange name, Stock name, Initial Capital, Date Range.
2. Entry Logic: The specific condition to open a position. Explicitly state if it uses today's or yesterday's data.
3. Exit Logic: The specific condition to close the position. Explicitly state if it uses today's or yesterday's data.
4. Metric: The specific performance indicator to calculate.

5. Output Format (CRITICAL)

You must output a single JSON object. Do not include markdown formatting (like ```)

The JSON key containing the narrative must be exactly matches the expected schema.

Example Structure:

```
{"nlp_strategy": "YOUR GENERATED TEXT HERE"}
```

6. Few-Shot Examples (Strictly Follow JSON Format)

Bad Example (Do NOT do this - Pure Text):

"We want to backtest Stock X... Sell if the 60-day Bias is above 10."

Example:

```
{"nlp_strategy": "We want to backtest Shanghai Stock Exchange-listed Stock X from Jan 1, 2020, to Dec 31, 2022, with an initial capital of CNY 1,000,000. For this test, the strategy should open a position whenever yesterday's closing price exceeds the 20-day Moving Average (calculated using data up to yesterday). Conversely, please execute a sell if the price falls back below this average. Finally, please calculate the Sharpe Ratio."}
```

Figure 9: Prompt for Converting Python Code to Natural Language.

Prompt for Evaluating Natural Language Strategies

You are a senior quantitative backtesting engineer (Python expert) with 10 years of experience. Your task is to conduct a strict Code Review.

Input Data Structure

You will receive three parts of input:

1. Base Rules: Hard constraints of the backtesting framework (e.g., lot size, execution price, calculation standards), which are considered absolute truth and must not be questioned.
2. Trading Strategy: Specific trading ideas defined by the user (e.g., signal triggering conditions, parameter settings).
3. Code Implementation: The Python script written based on the above two points.

Your Task: Dual-Dimension Audit

Please verify the following two dimensions item by item and generate a JSON audit report.

Dimension 1: Code Fidelity

Core Goal: Does the code 100% faithfully reproduce the [Base Rules] and [Trading Strategy]?

Review Checklist:

1. Parameter Consistency: Do values such as `window\_size`, `threshold`, and `cash` correspond exactly to the text?
2. Definition Precision: If the [Trading Strategy] is vague (e.g., writing only "moving average" without specifying "N-day", or "price" without specifying "closing price"), treat it as an implementation error ('False'). The code must not "hallucinate" undefined logic.
3. No Hidden Logic: Does the code contain extra logic not mentioned in the text (e.g., secretly added stop-loss, filters)? If so, treat it as an error.
4. Future Function Detection: Check if future data is used when calculating the signal for the current day (Look-ahead Bias).

Dimension 2: Strategy Validity

Core Goal: Is the [Trading Strategy] itself valid regarding quantitative principles? (Note: Do not evaluate the [Base Rules])

Review Checklist:

1. Executability: Can the signal trigger point be executed in the real market? (e.g., requiring a buy at today's highest price, which is unpredictable in real-time).
2. Logical Closed Loop: Are opening and closing conditions complete? Are there logical loopholes like "buy only, never sell" or "infinite loops"?
3. Causal Logic: Is there reverse causality? (e.g., using position results to infer buy signals).

Few-Shot Examples (Strictly Follow Logic)

Example 1: Mismatch in Parameters (Code False)

Strategy says "20-day MA", Code uses `rolling(30)`.

```
{  "code_correct": false,
   "code_correct_reason": "Parameter mismatch: Strategy specifies a 20-day MA, but code implements `window=30` in `rolling()` function.",
   "strategy_reasonable": true,
   "strategy_reasonable_reason": "Trading strategy is basically reasonable."}
```

.....

Base Rules

- Trading Mechanism (Execution): This study adopts a Long-only mechanism. Buy and sell operations strictly alternate; simultaneous buying and selling in a single trading day is prohibited. Trades are executed as Buy at Open, Sell at Close. No buying is performed on the last day of the backtest period, and positions are forced closed at the closing price.
- Buy/Sell Constraints: Single buy quantity is `int(cash balance // buy price)` (rounded down to an integer and at least 100 shares); Full Liquidation is strictly executed when selling. Commission fees and income tax are not considered during trading.
- Capital Management (Allocation Schemes): Fixed principal mode: Initial principal, no leverage. Buy quantity is dynamically calculated = `(cash balance // buy price)` (subject to the minimum lot size limit of the Constraints).
- Performance Evaluation (Metrics): Performance evaluation uses the mark-to-market method. Daily return is calculated based on the percentage change in the total portfolio value. Total value includes the closing market value of held assets and cash balance. Total value at today's close is `(held quantity \* today's closing price + cash balance)`. Annualized return is calculated based on 252 trading days. Empty positions (cash holding periods) must be taken into account when calculating indicators.
- Look-ahead Bias Control: In daily frequency trading strategies, data such as today's high, low, close, volume, turnover, and percentage change are only available after the market closes. To avoid look-ahead bias, factor calculations in this strategy strictly use historical data from T-1 and before, excluding any real-time information from day T, ensuring logic consistency between backtesting and live trading.
- Trading Pricing Standard: Since the timing of the highest and lowest prices is uncertain and can only be confirmed at the close, they are not actionable in practice. Considering this strategy uses daily frequency data (no intraday information), our strategy's buy and sell pricing standards rely on these two prices (Open and Close).

Figure 10: Prompt for Evaluating Natural Language Strategies

Prompt for Summarizer

Role Definition

You are a Quantitative Factor Analysis Expert. Your task is to analyze a trading strategy description and extract all factor names, and KPI names mentioned.

Task Instructions

1. Analyze the user's strategy description carefully.
2. Identify every technical indicator (e.g., moving averages, RSI, volatility) and performance metric (e.g., Sharpe Ratio, Max Drawdown).
3. Standardize & Extract the factor names using the following rules:
 - Full Names: Convert abbreviations to full English names (e.g., use "Rate of Change" instead of "ROC", "Moving Average" instead of "MA").
 - Time Windows: Always put the time window at the beginning (e.g., "20-day", "12-day").
 - Format: Use the format "[Time Period]-[Full Factor Name]" (e.g., "20-day Moving Average").
 - Consistency: If the text mentions a specific parameter (like "ROC6"), extract it as "6-day Rate of Change".
4. Output the result as a strictly valid JSON object containing a single key "factors".

Output Format Rules

```
{format_instructions}
```

Few-Shot Examples

Example 1

Input:

"We need to determine the optimal threshold value for the volume-based buy condition in the combined strategy for Yunnan Copper listed on the Shenzhen Stock Exchange. Please conduct a backtest from March 12, 2021, to August 7, 2023, with an initial capital of CNY 4,434,000. The strategy opens a position only when both of the following conditions are satisfied based on yesterday's (1) yesterday's 6-day Rate of Change (ROC6) is positive, and (2) the rolling standard deviation of trading volume over the previous ten days up to yesterday exceeds its 20-day moving average by a factor of the variable threshold. Conversely, the strategy closes the position when both of the following conditions are satisfied based on yesterday's (1) yesterday's trading volume exceeds its 12-day moving average, and (2) yesterday's 12-day Rate of Change (ROC12) is negative. The candidate threshold values to be tested are 1.8 and 2.0. Please calculate the Profit/Loss Ratio for each scenario and identify the threshold value that yields the maximum ratio."

Output:

```
{"factors": ["6-day Rate of Change", "10-day Standard Deviation of Trading Volume ", "12-day Moving Average of Volume", "12-day Rate of Change", "Profit Loss Ratio"]}
```

...

Figure 11: Factor Retrieval Prompt.

Prompt for Retriever

```

### Role
You are a Data Engineer and SQL Specialist.
Your ONLY task is to write a simple SQL SELECT query that retrieves the RAW market data required for a backtest.
You are NOT asked to implement the strategy logic in SQL, and you MUST NOT compute indicators in SQL.

### Critical Requirement (Read Carefully)
- Output ONLY the raw data rows needed for downstream backtesting in Python.
- Do NOT calculate indicators (MA/EMA/ROC/MACD/Sharpe/Volatility/etc.) in SQL.
- Do NOT translate entry/exit rules into SQL conditions.
- Do NOT use WITH / CTE / WITH RECURSIVE.
- Do NOT use window functions (OVER, LAG, AVG OVER, STDDEV OVER, etc.).
- Do NOT use ORDER BY: The execution tool AUTOMATICALLY sorts the result by `trade_date` ASC.
- Keep the SQL as a plain data extraction query: SELECT columns FROM one table WHERE filters.

### Database Context & Schema (STRICT VALIDATION)
You have access to a PostgreSQL database with 3 tables: `shanghai_stock_exchange`, `beijing_stock_exchange`, `shenzhen_stock_exchange`
ALLOWED Database Columns (Whitelist):
You can ONLY select columns from this list. If a word is not here, it is NOT a column.
- `trade_date`
.....

### Tool Usage & Execution Context
You have access to a tool named `executePostgresQuery`.
- Input: A standard SQL SELECT statement.
- Auto-Processing: The tool will automatically:
  1. Execute your query.
  2. Convert the result into a Pandas DataFrame.
  3. Sort the data by `trade_date` (ascending) automatically.
  4. Generate a text preview.
- Return Value:
  - If `status` is "success": It returns a text preview of the sorted data (first 5 and last 5 rows) and the total row count.
  - If `status` is "error": It returns the specific error message.
- TOOL USAGE LIMIT: You use tools at most 25 times.

### Data Mapping: Short Codes to Database Columns
The user strategy contains formulas with Short Codes. You must map them to the ALLOWED columns.
1. Column Mapping Table (Map these codes to SQL columns):
| Short Code in Formula | Required Database Column |
|-----|-----|
| `OPEN` | `opening_price` |
|.....

2. Ignored Short Codes (NOT Columns):
The following terms represent variables or functions, NOT database columns. Do NOT select them:
- `CASH`, `POSITION`, `PORTFOLIO`
- `DELAY`, `SMA`, `EMA`, `MAX`, `MIN`, `STD`, `MEAN`, `SQRT`, `ABS`, `CUMMAX`
Example Logic:
- If formula is `(CASH + POSITION * CLOSE) / DELAY(PERCENTAGE_CHANGE,1)`:
  - `CASH` -> Ignore.
  - `POSITION` -> Ignore.
|.....

### Date Range Rule
Strictly use the start and end dates provided in the user's strategy description.
- Do NOT extend the start date backwards.
- Do NOT add any extra buffer.

### Workflow Instructions
1) Identify the correct table based on exchange.
2) Match Company Name Smartly: Use English full names (e.g., 'Shenzhen Anche Technologies').
3) Parse Formulas: Look at the "Factors & Indicators" section. Map Valid Codes -> DB Columns. Ignore Variables (CASH/POSITION).
4) Produce ONE simple SELECT query to fetch these raw columns.
5) Call `executePostgresQuery` to verify.
6) When verified, output ONLY the final JSON.

### Final Output Protocol
Output ONLY a JSON object:
{"sql_statement": "YOUR_VERIFIED_SQL_HERE"}

```

Figure 12: Prompt For Retriever.

Prompt for Coder

```
# Role
You are a quantitative trading strategy backtesting expert specialized in Python and Pandas operations. Your goal is to accurately implement the user's trading strategy, run the backtest, and return the precise result in JSON format.

# ENVIRONMENT & DATA (CRITICAL)
- PRE-LOADED DATA: A pandas DataFrame named 'df' is ALREADY loaded in your Python environment.
- NO FILE I/O: DO NOT attempt to read any CSV, JSON, or other files. 'pd.read_csv()' will fail.
- DIRECT ACCESS: You must access 'df' directly in your code (e.g., 'df.head()', 'df['closing_price']').
- DATA ASSUMPTION: Do not consider missing or disordered trading days (assume data is continuous and ordered).

# Data preview:
{dataframe.head()}

# Task Workflow (MUST FOLLOW)
1. CODE GENERATION:
- Input Analysis: Read the Strategy Description (Natural Language) and the provided Factors (Short Code format).
- Indicator Translation: Convert the provided "Short Code" factors into Pandas Series by referencing the "Translation Table".
- Logic Construction: Translate the natural language rules (e.g., "Buy when SMA > Close") into Python logical conditions using the calculated Pandas indicators.
- Backtest Loop: Wrap this logic into a backtesting script that strictly adheres to the "Trading Rules" below.
2. EXECUTE & VERIFY:
- Run Code: Execute the script using the 'python_repl_ast' tool.
- Validation: Check if the output directly answers the user's specific question (e.g., a specific metric value or stock name).
- Self-Correction: If the execution fails or yields invalid results (like NaN), debug and re-run within the same turn.
3. OUTPUT JSON:
- Extract the final answer from the successful execution.
- Format the result strictly as a JSON object.
- DO NOT include any code, explanations, or reasoning in the final response.

# TRADING RULES & PROTOCOLS (STRICT ENFORCEMENT)
### 1. Execution Mechanism (Long-Only)
- Type: Long-only (no short selling).
- Alternation: Buy and Sell must strictly alternate. Simultaneous buying and selling in a single trading day is PROHIBITED.
- Prices:
  - Buy at Opening Price ('Price_open').
  - Sell at Closing Price ('Price_close').
- Last Day Rule: On the final day of the backtest period, DO NOT Buy. If holding a position, force a Full Liquidation at the Closing Price.
### 2. Constraints (Buy & Sell)
- Buy Quantity (Q): Must be an integer greater than or equal to 100 (e.g., 101 shares are allowed).
- Sell Quantity: Strict Full Liquidation (sell all shares held).
- Costs: Transaction fees and income taxes are NOT considered (0 cost).
### 3. Allocation Schemes (Money Management)
- Mode: Fixed Capital Mode (Initial Capital = 'Cash_0'). No leverage.
- Dynamic Sizing:
  -  $SQ_t = \text{floor}(\text{Cash}_t / \text{Price\_open}_t)$ 
  - Subject to the minimum size constraint (must be  $\geq 100$  shares).
### 4. Look-ahead Bias Control
- Principle: In daily trading strategies, data such as High, Low, Close, Volume, Amount, and Change are only known after the close.
- Constraint: Factor calculations MUST strictly use historical data from T-1 day and prior, with ONE exception.
- Exception: You ARE ALLOWED to use the current day's Opening Price ('Price_open_t') for signal generation, as it is known at the time of execution.
- Prohibition: Factor calculations must NOT contain Day T's Close, High, Low, Volume, or Amount.
### 5. Performance Metrics (Mark-to-Market)
- Method: Daily Mark-to-Market.
- Daily Return ( $Sr\_tS$ ):
  - Formula:  $Sr\_t = (PV\_t - PV_{\{t-1\}}) / PV_{\{t-1\}}$ 
  - Where  $SPV\_tS$  (Portfolio Value) = (Holding Quantity * Close Price_t) + Cash Balance_t.
- Metric Scope: Metrics must account for periods of empty positions (cash only).
- Annualized Return: Calculated based on a benchmark of 252 trading days.
- Risk-Free Rate: Fixed value of 0.0001 (daily).
### 6. Pricing Standard
- Criterion: Since High and Low prices have uncertain timing and are only confirmed at close, they are not operable. Trading execution is strictly based on Open and Close prices.

# STRICT CONSTRAINTS
1. NO GUESSING: You cannot answer questions like "which stock" or "what parameter" without running code to verify data.
2. SINGLE TOOL CALL: Combine all logic (indicators, signals, backtest loop) into ONE Python block.
3. PRINT RESULTS: Your Python code MUST 'print()' the final result so you can read it.
4. NO CODE IN ANSWER: The final response must ONLY be the JSON.
5. TOOL USAGE LIMIT: You use tools at most 25 times.
```

Figure 13: Prompt For Coder.